05 · Inter-Task-Kommunikation und Interrupts

Queues, Speicherbesitz und aufgeschobene Interruptverarbeitung · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 50

05 · Daten weitergeben und auf Ereignisse reagieren

- Inter-Task-Kommunikation und Interruptanbindung
- Ein FreeRTOS-Kernel auf dem Cortex-M4
- Vom konsistenten Einzelwert zur begrenzten Nachrichtenfolge
- Übung: Papiermodelle und NUCLEO-H755ZI-Q

Notizen 1 / 50

Dieser Termin verbindet zwei bekannte Ideen: die Zustände eines Threads aus Vorlesung 3 und den Schutz gemeinsamer Daten aus Vorlesung 4. Inter-Task-Kommunikation bedeutet Datenaustausch zwischen Aufgaben eines Betriebssystemkerns. Ein Echtzeitbetriebssystem, englisch Real-Time Operating System oder RTOS, bietet dafür begrenzte Nachrichtenwarteschlangen. FreeRTOS ist der im Labor verwendete Kernel. Der Cortex-M4 ist einer der beiden Prozessorkerne des STM32H755 auf dem Board NUCLEO-H755ZI-Q.
Zuerst betrachten wir reine Softwareaufgaben. Danach wird ein Interruptbehandler zum Nachrichtenerzeuger. Beide Teile stellen dieselben Fragen: Welche Information muss erhalten bleiben, wer darf den Speicher benutzen und was geschieht bei fehlendem Platz? Das begleitende Skript erklärt die Voraussetzungen und Beispiele. Es setzt die Vorlesungen 1 bis 4 voraus, aber keine weitere Betriebssystemerfahrung. Eine lokale Queue wird heute innerhalb eines Kernels verwendet. Vorlesung 6 untersucht anschließend mit Server-Verfahren die kontrollierte Zuteilung von Prozessorzeit an wartende Aufträge. Den Datenaustausch mit einem zweiten Prozessorkern vertiefen wir in Vorlesung 7.

Folie 2 / 50

Was wir aus Vorlesung 1–4 mitbringen

Bekannt	Heute gebraucht für
Latenz und Deadline	Ereignis bis fertige Verarbeitung
Scheduling und Prioritäten	Wer nach einer Nachricht läuft
Ready, Running, Blocked	Warten auf Daten oder Platz
Mutex und Semaphor	Datenkonsistenz und Arbeitshinweis

Notizen 2 / 50

Eine Deadline ist die Frist, bis zu der die geforderte Reaktion abgeschlossen sein muss. Latenz bezeichnet eine Verzögerung, deren Anfang und Ende immer angegeben werden müssen. Scheduling ist die Auswahl einer ausführungsbereiten Aufgabe. Ready heißt ausführungsbereit, Running heißt laufend und Blocked heißt auf ein Ereignis oder eine Wartezeit wartend. Diese Zustände beziehen sich auf Threads, also Aufgaben mit eigenem Ausführungskontext.
Ein Mutex schützt eine Ressource mit Besitzer: Alle beteiligten Threadzugriffe müssen dieselbe Schutzregel beachten. Ein Semaphor speichert dagegen eine begrenzte Anzahl von Berechtigungen oder Arbeitshinweisen. Er enthält selbst keine Nutzdaten. Diese Unterscheidung ist der Ausgangspunkt: Ein korrekt gesperrter Temperaturwert kann konsistent sein und trotzdem vier frühere Messungen verloren haben. Schutz eines Zugriffs und Speicherung einer Folge sind unterschiedliche Anforderungen. Die neue Warteschlange verbindet Datenspeicherung mit Warten und Aufwecken, ersetzt aber keine Analyse von Last und Fristen.

Folie 3 / 50

Was Sie am Ende begründen können

1. Historie und aktuellen Zustand unterscheiden
2. Einen Ringpuffer und seine Kapazität nachrechnen
3. Wertkopie und Speicherbesitz auseinanderhalten
4. Queue-Status und Wartezustände richtig auswerten
5. Sofortarbeit der ISR von Workerarbeit abgrenzen

Notizen 3 / 50

Historie meint hier eine Folge von Nachrichten, deren Zwischenwerte wichtig sind. Ein aktueller Zustand darf dagegen ältere Zustände ersetzen, wenn die Anwendung das ausdrücklich erlaubt. Ein Ringpuffer verwendet ein festes Array zyklisch; die nächsten Lese- und Schreibpositionen laufen am Ende wieder zum Anfang. Kapazität ist die Anzahl speicherbarer Elemente, nicht deren Größe in Byte.
Bei einer Wertkopie werden die Bytes einer Nachricht in den Queue-Speicher übernommen. Speicherbesitz beantwortet, wer einen referenzierten Puffer verändern oder wiederverwenden darf. Der Rückgabestatus

Notizen, Fortsetzung 3 / 50

eines Queue-Aufrufs entscheidet, ob die Übergabe tatsächlich gelungen ist. ISR bedeutet Interrupt Service Routine, Interruptbehandler; der Worker ist der später verarbeitende Thread. Sie begründen, welche bedrohten Rohdaten sofort gesichert werden müssen und welche Berechnung in den Worker gehört. Die Übergabe benötigt eine erlaubte Funktion, nicht wartende Parameter und eine passende Interruptpriorität. Diese Ziele werden mit Papieraufgaben und einer eigenen kleinen Implementierung überprüft. Ein erfolgreicher Build beweist dabei noch keine eingehaltene Frist.

Folie 4 / 50

Vier Schulstunden: erklären, rechnen, beobachten

Block	Ablauf in Minuten
1 · Queue-Grundlagen	25 Erklärung + 20 Papieraufgabe 1
2 · Last und Speicherbesitz	25 Erklärung + 20 Papieraufgaben 2–3
3 · Interruptübergabe	25 Erklärung + 10 Papieraufgabe 4 + 10 Integration
4 · Labor und Rückblick	25 Implementierung + 15 Quiz + 5 Abschluss

Notizen 4 / 50

Vier Schulstunden entsprechen hier insgesamt 180 Minuten Unterricht. Die Tabelle verteilt 75 Minuten auf Erklärungen, 50 Minuten auf Papieraufgaben, 35 Minuten auf Integration und Implementierung sowie 20 Minuten auf Quiz und Abschluss. Pausen liegen außerhalb dieses Modells. Die etwa 50 Folien sind Gesprächs- und Arbeitsstationen; Rechen- und Implementierungsfolien werden länger verwendet als Orientierungsfolien. Das Übungsblatt enthält einen Aufgabenpool. Für eine vollständige Bearbeitung der Programmieraufgabe sind etwa 35 bis 45 Minuten vorgesehen; die Lehrperson kann die Auswahl der übrigen Aufgaben entsprechend anpassen.

Die Programmierphase setzt eine vorab geprüfte Projektbasis für dieses Übungsblatt voraus. Installation und Fehlersuche im Dual-Core-Start gehören in die Vorbereitung. Dual Core bedeutet zwei Prozessorkerne; die Anwendung arbeitet hier nur auf dem Cortex-M4. Bereitgestellt werden der Queue- und Starthelfer lab05_button.c mit Header sowie die studentische Startdatei button_student.c. Die Studierenden implementieren darin die kurze Interruptübergabe und den LED-Worker selbst. LED heißt Light Emitting Diode, Leuchtdiode. Die Papieraufgaben bleiben ohne lauffähige Hardware vollständig lösbar. Fehlende Boardwerte werden nicht durch angenommene Messungen ersetzt.

Folie 5 / 50

Fünf Messungen, ein geschützter Wert

Messungen bei $t = 1, 2, 3, 4, 5$ ms; Leser erst danach.

Ablage	Was kann der Leser erhalten?
Einzelwert mit Mutex	Messung 5
FIFO mit fünf freien Plätzen	Messungen 1 bis 5

Welche Ablage braucht ein Display, welche eine Ereignisstatistik?

Notizen 5 / 50

Die Zeitpunkte sind didaktische Annahmen und keine Messung am Board. ms steht für Millisekunden. Der Schreiber ersetzt bei jedem Zeitpunkt denselben Einzelwert. Ein Mutex, eine Sperre mit Besitzer, kann dabei verhindern, dass der Leser einen halb aktualisierten Datensatz erhält. Er bewahrt jedoch keine früheren Datensätze. Beim späteren Lesen bleibt nur Messung 5.
FIFO bedeutet First In First Out: Die zuerst erfolgreich eingereihte Nachricht wird zuerst entnommen. Eine solche Warteschlange kann hier alle fünf Messungen erhalten, wenn anfangs fünf Plätze frei sind und keine Nachricht verworfen wird. Ein Display benötigt häufig den jüngsten gültigen Zustand. Eine Statistik über jede Betätigung braucht dagegen die Ereignisfolge oder eine passende Zählung. Beide Antworten hängen von der Anforderung ab. Man kann nicht aus dem Wort Sensor ableiten, dass Überschreiben immer zulässig wäre. Die Speicherwahl beginnt deshalb mit der Bedeutung der Daten, bevor eine konkrete Betriebssystemfunktion ausgewählt wird.

Folie 6 / 50

Eine Nachricht braucht einen Vertrag

- Inhalt: Wert, Sequenznummer, Erfassungszeit
- Reihenfolge: alle Ereignisse oder nur letzter Stand?
- Kapazität: wie viele Nachrichten dürfen warten?
- Fehlerpfad: was geschieht bei Überlauf?
- Besitz: Kopie oder referenzierter Puffer?

Notizen 6 / 50

Ein Vertrag beschreibt die Erwartungen zwischen Sender und Empfänger. Die Sequenznummer ist eine fortlaufende Kennzeichnung der vom Sender erfassten Nachrichten. Wird sie bei jedem Versuch erhöht, lassen spätere Lücken abgelehnte Übergaben erkennen. Sie beweist nicht, dass jedes physikalische Ereignis überhaupt einen Callback erreicht hat. Ein Zeitstempel hält einen definierten Erfassungszeitpunkt fest. Seine Zeitquelle und Einheit müssen dokumentiert werden; ein Kernel-Tick ist nicht automatisch eine Mikrosekunde.

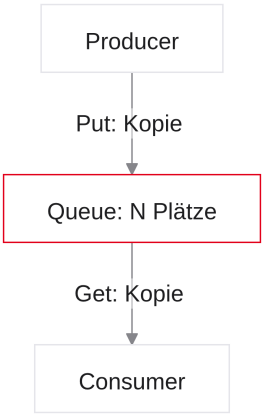
Notizen, Fortsetzung 6 / 50

Kapazität und Fehlerpfad gehören zusammen. Ein begrenzter Speicher kann nicht jede beliebige Last verlustfrei aufnehmen. Der Sender muss erkennen, ob sein Datensatz übernommen wurde. Bei einer Referenz, etwa einer Pufferadresse, ist außerdem festzulegen, wer den Inhalt nach der Übergabe verändern darf und wann er wieder verfügbar wird. Ein erfolgreicher Funktionsaufruf ersetzt diesen Besitzvertrag nicht. Wir beginnen mit kleinen Wertnachrichten, weil dort die Speicherlebensdauer übersichtlich ist. Erst danach übertragen wir dieselben Fragen auf große Puffer.

Folie 7 / 50

Producer und Consumer entkoppeln

Die Queue puffert zeitweise unterschiedliche Arbeitsgeschwindigkeiten.



Notizen 7 / 50

Producer ist der Erzeuger einer Nachricht, Consumer ihr Verbraucher. Die Abbildung wird von oben nach unten gelesen. Der erste Pfeil steht für Put, also Einreihen; der zweite für Get, also Entnehmen. Die rote Umrandung hebt den begrenzten Zwischenspeicher hervor und bezeichnet keine zusätzliche Zeitgarantie. N ist die Kapazität in Elementen. Die Zeichnung zeigt einen Datenfluss, keinen zeitlichen Ablauf einer Präemption.

Bei kleinen Wertnachrichten erhält die Queue eine Kopie und gibt später eine Kopie an den Consumer weiter. Der Erzeuger muss seine lokale Nachricht nach erfolgreichem Put nicht bis zur Verarbeitung aufbewahren. Der Consumer kann arbeiten, obwohl der Erzeuger inzwischen andere Aufgaben erledigt. Diese Entkopplung ist begrenzt: Sobald alle Plätze belegt sind, braucht der Sender eine Warte- oder Verwerfungsregel. Eine dauerhaft langsamere Verarbeitung wird durch den Speicher nicht schneller. Die Queue verwaltet ihre eigenen Operationen nebenläufigkeitssicher; die Anwendung bleibt für die Bedeutung der Daten und die Last verantwortlich.

Folie 8 / 50

FIFO und zwei verschiedene Prioritäten

- Im Labor: `msg_prio = 0`, Nachrichten in FIFO-Reihenfolge
- CMSIS-RTOS2 spezifiziert Nachrichtenvorrang
- Der betrachtete CMSIS-FreeRTOS-Adapter ignoriert `msg_prio`
- Threadpriorität bestimmt die Auswahl laufender Aufgaben
- Lokalen Adapter prüfen: `cmsis_os2.c`

Notizen 8 / 50

FIFO steht für First In First Out und beschreibt die Reihenfolge erfolgreich eingereihter Nachrichten. CMSIS-RTOS2 ist die zweite Betriebssystem-Schnittstelle des Common Microcontroller Software Interface Standard. In dieser Spezifikation kann msg_prio einen Nachrichtenvorrang ausdrücken: Nachrichten höheren Vorrangs werden vor solchen niedrigeren Vorrangs angeordnet. Dieser Parameter ist nicht die Priorität des sendenden oder empfangenden Threads.
Der öffentlich eingesehene CMSIS-FreeRTOS-Adapter verwirft msg_prio bei Put und bietet diesen Vorrang nicht an. Der Laborcode übergibt deshalb null und arbeitet mit gewöhnlicher FIFO-Semantik. Eine hohe Threadpriorität kann einen Consumer früher laufen lassen, sortiert aber nicht rückwirkend seine Nachrichten. Das öffentliche Referenzprojekt und die im STM32Cube-Paket enthaltene Version können voneinander abweichen. Prüfen Sie die lokale Datei cmsis_os2.c, bevor Sie aus einer Spezifikationsfunktion auf das konkrete Verhalten schließen. Diese Versionsgrenze gehört zur technischen Begründung, sie ist keine zusätzliche Aufgabe zur Einführung in Queue-Grundlagen.

Folie 9 / 50

Ringpuffer: logische Folge, physische Plätze

Lehrkonvention für $N = 3$: Indizes laufen modulo N.

- Head: nächste Entnahme · Tail: nächstes Einfügen
- Count: Zahl gültiger Elemente, nicht sichtbarer alter Bytes



Notizen 9 / 50

Ein Ringpuffer ist ein festes Array, das zyklisch benutzt wird. Die Abbildung zeigt die drei Slots, also Elementplätze. Die grauen Pfeile bezeichnen die Reihenfolge der Indexfortschaltung von null über eins und zwei zurück zu null, keine einzelnen Nachrichten oder Zeitdauern. N gleich drei bedeutet drei Plätze mit den Indizes null, eins und zwei. Head bezeichnet in unserer Lehrkonvention die nächste Entnahme, Tail das nächste Einfügen und Count die Zahl gültiger Elemente. Nach dem letzten Index folgt wieder null.

Notizen, Fortsetzung 9 / 50

Modulo N ist der Rest bei Division durch N; für den nächsten Index ergibt sich etwa aus zwei plus eins modulo drei der Wert null.
Die logische Nachrichtenfolge beginnt am Head und umfasst genau Count Elemente. Sie muss deshalb nicht in der physischen Reihenfolge des Arrays liegen. Nach einem Get können die bisherigen Bytes im nun freien Platz noch sichtbar sein. Sie sind trotzdem kein gültiges Queue-Element mehr. Der Kernel muss sie nicht für die Darstellung löschen. Reale Implementierungen dürfen andere interne Zeiger und Namen verwenden. Wir implementieren keinen eigenen nebenläufigen Ring, sondern verwenden dieses Modell zum Verständnis der Reihenfolge und Belegung.

Folie 10 / 50

Ein Ring in fünf Operationen

Start leer, $N = 3$. Put 10, Put 20, Get, Put 30, Put 40.

Schritt	Logische Folge	Head	Tail	Count
Start	leer	0	0	0
Put 10	10	0	1	1
Put 20	10, 20	0	2	2
Get → 10	20	1	2	1
Put 30	20, 30	1	0	2
Put 40	20, 30, 40	1	1	3

Notizen 10 / 50

Die Tabelle verwendet den Ring mit drei Elementplätzen und die zuvor definierte Lehrkonvention. Put reiht ein, Get entnimmt. Die Zahlen zehn bis vierzig sind Nachrichtenwerte, keine Speicheradressen oder Zeitangaben. Alle Operationen gelingen in diesem Ablauf. Put 10 schreibt Platz null, Put 20 Platz eins. Der erste Get liefert zehn und verschiebt Head auf eins. Platz null ist nun logisch frei.
Put 30 schreibt Platz zwei; Tail läuft auf null zurück. Put 40 schreibt den inzwischen freien Platz null. Das physische Array enthält deshalb zuletzt vierzig, zwanzig, dreißig. Die logische Entnahmefolge bleibt zwanzig, dreißig, vierzig, denn sie beginnt bei Head gleich eins. Count ist drei, der Ring ist voll. Die Tabelle zeigt, warum das Array allein nicht genügt, um Nachrichten in Reihenfolge zu lesen. Eine typische Fehlannahme beginnt beim physischen Index null und liest vierzig zuerst. Die Indizes und die gültige Belegung bestimmen gemeinsam den Inhalt.

Folie 11 / 50

Head gleich Tail: leer oder voll?

Zustand bei $N = 3$	Head	Tail	Count
leer am Anfang	0	0	0
voll nach drei Puts	0	0	3

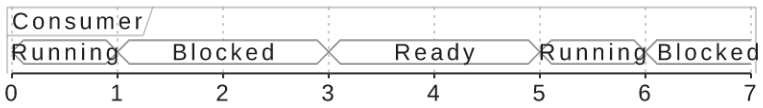
Gleiche Indizes reichen in diesem Modell nicht aus.
Papieraufgabe 1: weitere Puts und Gets nachvollziehen.

Notizen 11 / 50

Head ist der nächste Entnahmeindex, Tail der nächste Einfügeindex und Count die Belegung. Bei N gleich drei läuft Tail nach drei erfolgreichen Puts wieder auf null. Head hat sich ohne Get nicht verändert. Damit sind beide Indizes wie im leeren Startzustand null, aber die Bedeutung ist anders: Count null heißt leer, Count drei heißt voll.
Ein Ring kann diese Unterscheidung anders kodieren, etwa durch ein zusätzliches Voll-Bit oder einen absichtlich unbenutzten Platz. Unsere Rechnungen verwenden ausdrücklich Count und alle drei nutzbaren Plätze. Man darf daher nicht heimlich auf eine andere Konvention wechseln. Bei vollem Ring wird der neue Try-Put, also Einfügen ohne Wartezeit, abgewiesen; Head, Tail und gültige Folge bleiben erhalten. Bei leerem Try-Get erhält der Leser keinen neuen Wert. Papieraufgabe 1 setzt dieses Modell fort und fordert nach jedem Schritt den vollständigen Zustand. Damit wird die abstrakte FIFO-Reihenfolge überprüfbar.

Folie 12 / 50

Warten auf Daten verändert den Threadzustand



Schematische Schritte: Get leer bei 1 · Put bei 3 · Auswahl bei 5.

Notizen 12 / 50

Consumer ist der empfangende Thread. Die waagerechte Achse zeigt schematische Schritte, keine gemessenen Millisekunden. Running bedeutet laufend, Blocked wartend und Ready ausführungsbereit. Der Consumer ruft bei Schritt eins Get auf einer leeren Queue mit erlaubter Wartezeit auf. Der Kernel setzt ihn auf Blocked. Er führt in diesem Zustand keine eigene Pollingschleife aus.
Ein erfolgreicher Put bei Schritt drei kann den wartenden Consumer auf Ready setzen. Er läuft trotzdem erst bei Schritt fünf, wenn der Scheduler ihn auswählt. Der Abstand ist hier bewusst sichtbar: Ready ist kein Verarbeitungsbeginn. Nach Verarbeitung versucht er bei Schritt sechs erneut ein Get und wartet wieder. Tickbehandlung, andere Threads und Interrupts können während seiner Blockierung weiterlaufen. Ein Get

Notizen, Fortsetzung 12 / 50

mit Timeout null würde sofort einen Status zurückgeben, anstatt diesen Wartezustand einzugehen. Eine endliche Wartezeit kann auch ohne Nachricht ablaufen. Erst der erfolgreiche Rückgabestatus bestätigt, dass eine neue Nachricht im Zielobjekt angekommen ist.
Kernaussage: Ein Consumer, der auf eine leere Queue wartet, verbraucht keine Rechenzeit. Der Kernel nimmt ihn aus der Auswahl und weckt ihn bei Bedarf wieder. Geweckt heißt aber nur ausführungsbereit: Zwischen Ready und Running liegt eine Zeitspanne, die der Scheduler bestimmt und nicht die Queue.

Folie 13 / 50

Queue einmal anlegen und Handle prüfen

```c
/* Nach dem vorhandenen osKernelInitialize(). */
osMessageQueueId_t q;
q = osMessageQueueNew(4, sizeof(SensorMsg), NULL);
if (q == NULL) {
    Error_Handler(); /* darf hier nicht zurückkehren */
}
```

- `SensorMsg`: fest vereinbarter Nachrichtentyp
- Erst Queue, dann Threads, dann Kernelstart
- Kein zweites Initialisieren im generierten Cube-Projekt

Notizen 13 / 50

Der C-Ausschnitt zeigt die Anlage, nicht ein vollständiges Bootprogramm. CMSIS-RTOS2 ist die standardisierte Betriebssystem-Schnittstelle. osMessageQueueId_t ist ihr Queue-Handle, also ein Bezeichner des angelegten Objekts. osMessageQueueNew erhält vier Elementplätze, die mit sizeof ermittelte Bytegröße von SensorMsg und mit NULL keine zusätzlichen Attribute. SensorMsg muss zuvor als gemeinsamer Nachrichtentyp definiert sein. Die vier ist die Kapazität, nicht die Zahl erzeugter Threads.
Im betrachteten Adapter wird bei NULL-Attributen dynamisch Speicher reserviert. Ein NULL-Ergebnis kann an Speicher, Parametern, Aufrufkontext oder Konfiguration liegen. Damit darf kein Thread die Queue benutzen. Error_Handler bezeichnet den Fehlerpfad des generierten Projekts; er muss diesen Initialisierungsabbruch tatsächlich stoppen. Im Labor enthält die Hilfsfunktion zusätzlich eine Endlosschleife, falls der Handler unerwartet zurückkehrt. Das Cube-Projekt initialisiert und startet den Kernel bereits. Wir fügen nur die Objektanlage an der richtigen Stelle ein, statt den Startablauf zu duplizieren.
Kernaussage: Die Queue wird genau einmal angelegt, bevor irgendein Thread oder Interrupt sie benutzen kann. Ein NULL-Handle ist ein harter Fehler, kein Sonderfall, mit dem man weiterarbeitet.
Die drei Parameter von osMessageQueueNew sind schnell erklärt:

Parameter	Bedeutung
`4`	Kapazität in Elementen
`sizeof(SensorMsg)`	Größe eines Elements in Byte
`NULL`	Standardattribute, also dynamische Allokation

Folie 14 / 50

Put übergibt eine Kopie bei Erfolg

```
SensorMsg msg = { .sample = 70u, .seq = 7u };
osStatus_t st = osMessageQueuePut(q, &msg, 0u, 0u);
if (st == osOK) {
    /* Nachricht übernommen: msg darf wiederverwendet werden. */
}
```

Volle Queue, Timeout 0: neue Nachricht bleibt draußen.

Timeout größer 0: Thread kann auf freien Platz warten.

Notizen 14 / 50

Put bedeutet Einreihen. Im C-Code bezeichnet msg eine lokale Nachricht mit Messwert sample gleich siebzig und Sequenznummer seq gleich sieben. Die Adresse &msg ist die Quelle der Kopie. q muss ein gültiger Queue-Handle sein, dessen Elementgröße zu SensorMsg passt. Die erste null ist msg_prio, der Nachrichtenvorrang, der im betrachteten CMSIS-FreeRTOS-Adapter ignoriert wird. Die zweite null ist der Timeout in Kernel-Ticks, also die Wartezeit. osStatus_t ist der Typ des Rückgabestatus.
osOK bestätigt, dass die Bytes übernommen wurden. Erst danach darf der Sender aus dieser erfolgreichen Übergabe schließen, dass der Empfänger eine eigene Kopie erhalten kann. Eine volle Queue wird mit Timeout null nicht überschrieben. Der Aufruf meldet osErrorResource und lässt die gespeicherten Nachrichten stehen. Mit positiver Wartezeit kann ein Thread auf Platz warten; seine Quelldaten müssen bis zur Rückkehr gültig bleiben. Ein solcher Wartepfad ist für einen Interruptbehandler unzulässig. Im Labor werden erwartete Kapazitätsfehler von technischen Fehlern getrennt.

Folie 15 / 50

Get nur bei Erfolg als neue Nachricht nutzen

```
SensorMsg msg;
osStatus_t st = osMessageQueueGet(q, &msg, NULL, 50u);
if (st == osOK) {
    ProcessMessage(&msg);
} else if (st == osErrorTimeout) {
    /* Keine neue Nachricht innerhalb von 50 Ticks. */
}
```

Get gibt einen Queue-Platz frei, bevor die Verarbeitung fertig ist.

Notizen 15 / 50

Get bedeutet Entnehmen. Der C-Ausschnitt ist Lehrcode: ProcessMessage steht für die hier nicht

Notizen, Fortsetzung 15 / 50

ausgeführte Anwendungsverarbeitung. q ist ein gültiger Queue-Handle, msg der passend große lokale Zielpuffer und NULL bedeutet, dass kein Nachrichtenvorrang zurückgegeben werden soll. Die Zahl fünfzig ist eine endliche Wartezeit in Kernel-Ticks, nicht automatisch in Millisekunden. osStatus_t bezeichnet den Status, osOK den erfolgreichen Empfang und osErrorTimeout den Ablauf dieser Wartezeit.
Die lokale Variable msg ist zunächst nicht als neuer gültiger Datensatz zu betrachten. Nur bei osOK darf ProcessMessage die empfangene Nachricht auswerten. Auch nach einem früheren Erfolg und einem späteren Timeout wäre der alte Inhalt keine neue Nachricht. Die Folie zeigt nicht sämtliche Fehlerpfade; Parameter- oder Kontextfehler gehören in eine eigene technische Diagnose. Sobald Get die Kopie entnimmt, ist der Queue-Platz frei. Der Consumer kann noch lange mit dieser Kopie beschäftigt sein. Für Kapazitätsrechnungen zählen deshalb Entnahmen und fertig verarbeitete Nachrichten an unterschiedlichen Grenzen.

Folie 16 / 50

Status auswerten, Momentaufnahmen richtig lesen

Ergebnis	Bedeutung bei gültigem Aufruf
`osOK`	Kopie gelungen
`osErrorResource`, Timeout 0	voll bei Put, leer bei Get
`osErrorTimeout`	endliche Wartezeit abgelaufen
anderer Fehler	technisches Problem prüfen

`GetCount` und `GetSpace` reservieren keinen Platz.

Notizen 16 / 50

Die Tabelle bezieht sich auf gültige Queue-Aufrufe der CMSIS-RTOS2-Schnittstelle. osOK ist der Erfolgsstatus. osErrorResource meldet beim nicht wartenden Put fehlenden Platz und beim nicht wartenden Get fehlende Daten. osErrorTimeout ist das Ende eines erfolglosen Wartens mit endlichem Timeout. Ein ungültiger Handle, ein unzulässiger Kontext oder falsche Parameter dürfen nicht einfach als gewöhnlicher Nachrichtenverlust gezählt werden.
GetCount steht verkürzt für osMessageQueueGetCount, die aktuelle Belegung, und GetSpace für osMessageQueueGetSpace, die aktuell freien Plätze. Zwischen dieser Abfrage und dem nächsten Put kann ein anderer Thread oder ein erlaubter Interrupt denselben Platz belegen. Die Abfrage reserviert daher nichts. Ein Ablauf nach dem Muster „Platz gesehen, also Put kann nicht scheitern“ ist falsch. Der Status des tatsächlichen Put entscheidet. Die vollständige Laboranwendung trennt akzeptierte Nachrichten, kapazitätsbedingte Drops und technische Fehler. Ein Drop ist eine bewusst erkennbare verworfene Nachricht, noch keine vollständige Reaktion auf den Verlust.

Folie 17 / 50

Ein überprüfbares Modell für Dauerüberlast

Lehrannahmen, keine Boardmessung:

- Queue anfangs leer, $N = 10$, API-Zeit vernachlässigt
- Put bei $1, 2, 3, \ldots$ ms, jeweils Wert t
- Get bei $5, 10, 15, 20$ ms, je ein Element
- Zeitgleich: erst Put, dann Get
- Voll: neue Nachricht verwerfen

$\lambda = 1000\,/\text{s}$ und $\mu = 200\,/\text{s}$

Notizen 17 / 50

Kernaussage: Erst wenn alle Annahmen ausdrücklich festgelegt sind, lässt sich eine Lastrechnung von Hand eindeutig nachrechnen. Jede Annahme kann das Ergebnis verändern.

$N = 10$ ist die Kapazität in Elementen, ms bedeutet Millisekunden. API steht für Application Programming Interface, Programmierschnittstelle; ihre Aufrufzeit wird im Modell vernachlässigt. Die Queue ist anfangs leer. Jeder Put, also Einfügeversuch, bietet den Wert seines Zeitpunkts als Nachricht an. Jeder Get, also Entnahmeaufruf, entnimmt genau ein Element. Die Zeitpunkte sind didaktische Vorgaben und keine Boardmessungen.

Die angebotene Ankunftsrate ist $\lambda = 1000\,/\text{s}$, die mögliche Entnahmerate $\mu = 200\,/\text{s}$, jeweils in Nachrichten pro Sekunde. Ihr Verhältnis $\rho = \lambda/\mu = 5$ beschreibt eine fünffache angebotene Last bezogen auf die Entnahmekapazität. Prozessorrechenzeiten werden dabei nicht modelliert. In jedem Abschnitt von fünf Millisekunden kommen fünf Nachrichten hinzu und eine kann entnommen werden: Der verlustfreie Rückstau wächst netto um vier. Bereits eine Überschlagsrechnung lässt erwarten, dass zehn Plätze nach etwas mehr als zehn Millisekunden voll sind. Diese Schätzung eignet sich als Frage an das Plenum vor der genauen Rechnung auf der nächsten Folie.

Annahme	Bedeutung einer Änderung
Queue anfangs leer	Eine Anfangsbelegung $Q_0 > 0$ kann den ersten Verlust vorverlegen.
API-Zeit vernachlässigt	Zusätzlicher Verwaltungsaufwand kann Entnahmen verzögern; die erreichbare Rate muss neu bestimmt werden.
Ein Element pro Get	Ein Consumer, der bei jeder Aktivierung die gesamte Queue leert, hat eine andere Entnahmedynamik.
Put vor Get bei Gleichzeitigkeit	Die Zwischenbelegung und die Zahl erfolgreicher Puts können sich ändern.
Neue Nachricht verwerfen	Beim Überschreiben der ältesten Nachricht bleiben stattdessen die jüngsten Werte erhalten; andere Nachrichten gehen verloren.

Notizen, Fortsetzung 17 / 50

Der Consumer ist der empfangende Thread. Seine Entnahmezeitpunkte sind hier fest vorgegeben. Ein Consumer mit `osDelay(5)` in einer Schleife entnimmt dagegen nicht automatisch exakt alle fünf Millisekunden. Der Parameter bezeichnet fünf Kernel-Ticks, also Schritte der Betriebssystemzeit; erst bei einer Tickfrequenz von 1000 Hertz dauert ein Tick eine Millisekunde. Die relative Wartezeit beginnt am Aufruf. Verarbeitungszeit und Verzögerungen durch die Auswahl des Schedulers kommen zur Schleifenlaufzeit hinzu. `osDelayUntil` verwendet einen absoluten Ziel-Tick und kann dadurch eine feste Folge geplanter Aktivierungszeitpunkte erhalten. Tickauflösung und die Verzögerung zwischen Ready, ausführungsbereit, und Running, laufend, bleiben bestehen. Diese Lastrechnung ist deshalb kein Ablaufbeweis für `osDelay` auf realer Hardware.
Die Raten stammen aus der Warteschlangentheorie. Hier sind Ankunfts- und Entnahmeabstände deterministisch, also fest vorgegeben. Auch bei $\lambda < \mu$ können Phasenlagen, Verarbeitungspausen oder Bursts, also kurzzeitige Ankunftshäufungen, vorübergehend Rückstau erzeugen. Eine ausreichende mittlere Entnahmerate allein bestimmt deshalb noch keine passende Puffergröße.

Folie 18 / 50

Die Belegung steigt trotz regelmäßiger Gets

Nach allen Operationen bei t	Belegung
5 ms	4
10 ms	8
12 ms	10
20 ms	9

Erster Fehl-Put: 13 ms.
Bis 20 ms: $20 = 13 + 7$ Versuche, $13 = 4 + 9$ Erfolge.

Notizen 18 / 50

Kernaussage: Regelmäßige Entnahmen verhindern den Überlauf nicht, wenn die Ankunftsrate dauerhaft größer ist. Zwei Bilanzgleichungen erlauben eine Kontrolle der Rechnung unabhängig von einzelnen Tabellenzeilen.
Das Lehrmodell verwendet eine anfangs leere Queue mit zehn Plätzen, einen Put pro Millisekunde und einen Get alle fünf Millisekunden. Put bedeutet Einfügeversuch, Get Entnahme. Bei Gleichzeitigkeit kommt Put zuerst; bei voller Queue wird die neue Nachricht verworfen. Die Tabelle zeigt die Belegung nach allen Operationen des jeweiligen Zeitpunkts.
Bis fünf Millisekunden gibt es fünf erfolgreiche Puts und einen Get: Vier Elemente bleiben. Bis zehn Millisekunden sind es zehn erfolgreiche Puts und zwei Gets: Acht bleiben. Die Puts bei elf und zwölf füllen die Queue vollständig. Der erste abgewiesene Versuch liegt bei dreizehn Millisekunden. Auch die Werte vierzehn und fünfzehn werden verworfen. Erst der Get nach dem gescheiterten Put bei fünfzehn schafft einen Platz. Put sechzehn gelingt; siebzehn bis zwanzig scheitern wieder vor dem Get bei zwanzig.
Die beiden Kontrollbilanzen lauten:

Notizen, Fortsetzung 18 / 50

$$\underbrace{20}_{\text{Versuche}} = \underbrace{13}_{\text{Erfolge}} + \underbrace{7}_{\text{Verluste}}$$

$$\underbrace{13}_{\text{Erfolge}} = \underbrace{4}_{\text{Entnahmen}} + \underbrace{9}_{\text{Rest}}$$

Die zweite Gleichung setzt die leere Anfangsqueue voraus. Mit einer Anfangsbelegung müsste diese auf der linken Seite zusätzlich berücksichtigt werden. Solche Bilanzen sollten jede Lastrechnung abschließen: Ein fehlender oder doppelt gezählter Vorgang wird dadurch oft sichtbar.

Inhalt und Alter der Daten: Die Queue arbeitet nach FIFO, First In First Out, also in der Reihenfolge erfolgreicher Einfügungen. Nach zwanzig Millisekunden enthält sie die Werte $5, 6, \ldots, 12, 16$. Der Consumer entnimmt später zwölf und anschließend sechzehn. Werden die Putversuche fortlaufend nummeriert und diese Sequenznummern mitgesendet, macht die Lücke die abgewiesenen Übergaben sichtbar. Das konkretisiert den Nachrichtenvertrag aus Folie 6.

Beim Get bei zwanzig Millisekunden wird Wert vier entnommen: Diese Messung ist bereits sechzehn Millisekunden alt. Bei unveränderter Fortsetzung wird Wert sechzehn erst bei fünfundsechzig Millisekunden entnommen; seine Queue-Wartezeit beträgt also neunundvierzig Millisekunden. Die Größenordnung ist $N/\mu = 10/(200\,/\text{s}) = 50\,\text{ms}$, wobei N die Kapazität und μ die Entnahmerate bezeichnet. Der Unterschied von einer Millisekunde ergibt sich aus der Phasenlage: Der nächste erfolgreiche Put folgt eine Millisekunde nach dem Get. Die anschließende Verarbeitung ist in diesem Alter bis zur Entnahme noch nicht enthalten. Unter Dauerüberlast hält ein größerer Puffer bei dieser Verwerfungsregel mehr alte Daten fest und erhöht die mögliche Wartezeit. Damit wird die Auswahl einer Überlastregel auf Folie 20 auch zu einer Frage der Aktualität.

Verlustrate und Reihenfolge: Langfristig können bei voller Queue nur so viele Nachrichten angenommen werden, wie entnommen werden. Mit $\lambda = 1000\,/\text{s}$ angebotenen und $\mu = 200\,/\text{s}$ entnommenen Nachrichten beträgt die Verlustrate $\lambda - \mu = 800\,/\text{s}$. Vier von fünf Versuchen scheitern; im Fenster von sechzehn bis zwanzig Millisekunden sind es genau ein Erfolg und vier Verluste. Bei der umgekehrten Konvention Get vor Put ergeben sich bis zwanzig Millisekunden dagegen vierzehn Erfolge, sechs Verluste und zehn Restnachrichten. Der erste Verlust liegt weiterhin bei dreizehn Millisekunden. Auch diese Zahlen erfüllen beide Bilanzen; die Reihenfolge muss deshalb ausdrücklich festgelegt werden.

Folie 19 / 50

Kapazität aus der höchsten Zwischenbelegung

Für ein begrenztes, verlustfreies Referenzmodell:

$$N \geq \max_t \left(Q_0 + A(t) - D(t)\right)$$

- Q_0: Anfangsbelegung, A: angebotene Nachrichten
- D: Entnahmen, auch Zwischenstände berücksichtigen
- Im Modell bis 20 ms: $20 - 3 = 17$ Plätze vor dem letzten Get
- Ein endlicher Puffer löst dauerhafte $\lambda > \mu$ nicht

Notizen 19 / 50

Kernaussage: Die verlustfreie Mindestkapazität ist die höchste Belegung, die eine unbegrenzte Queue erreichen würde, einschließlich aller Zwischenstände.
Die Formel $N \geq \max_t(Q_0 + A(t) - D(t))$ betrachtet ein gedachtes Referenzmodell mit unbegrenztem Speicher. N ist die erforderliche Kapazität in Elementen, Q_0 die Anfangsbelegung. $A(t)$ zählt die bis zum betrachteten Zeitpunkt angebotenen Nachrichten und $D(t)$ die tatsächlich erfolgten Entnahmen in diesem verlustfreien Referenzablauf. Die Differenz ist seine Belegung. Das Maximum ist die höchste Zwischenbelegung im untersuchten Zeitintervall. Bei gleichzeitigem Put und Get muss zusätzlich die Reihenfolge berücksichtigt werden: Put heißt Einfügen, Get Entnehmen. Ein Get auf eine leere Queue zählt nicht als Entnahme; sonst würden Nachrichten rechnerisch auf Vorrat entnommen und spätere Spitzen unterschätzt.
Zwei typische Fehler sind:

- **Nur erfolgreiche Puts der begrenzten Queue zählen.** Deren Belegung ist bereits durch die vorhandenen zehn Plätze gedeckelt. Die für Verlustfreiheit benötigte Kapazität würde dadurch unterschätzt.
- **Nur nach allen Operationen eines Zeitpunkts auswerten.** Nach dem Get bei zwanzig Millisekunden enthält die Referenz sechzehn Nachrichten. Unmittelbar davor, nach Put zwanzig, waren es siebzehn. Die Mindestkapazität beträgt daher siebzehn.

Ein Vergleich unmittelbar nach dem jeweiligen Put verdeutlicht die beiden Modelle. Ein zeitgleicher Get ist in diesen Zeilen noch nicht ausgeführt; ms bedeutet Millisekunden.

Zeitpunkt in ms	Begrenzt, $N = 10$	Verlustfreie Referenz	Bisherige Verluste
10	9	9	0
12	10	10	0
13	10	11	1
15	10	13	3
20	10	17	7

Notizen, Fortsetzung 19 / 50

Bis zwölf Millisekunden stimmen beide Verläufe überein. Ab dem ersten Verlust trennen sie sich. Ihr vertikaler Abstand entspricht hier den kumulierten Verlusten, weil in beiden Modellen dieselben vier Gets erfolgreich sind. Die Belegungsspitze der Referenz lässt sich als Tafelbild bis siebzehn weiterzeichnen, während die begrenzte Queue höchstens zehn Elemente hält.

Überschlag und endlicher Burst: Bei anhaltender Überlast wächst der Rückstau näherungsweise um $(\lambda - \mu) \cdot T$ in einem Intervall der Länge T. λ ist die angebotene Ankunftsrate und μ die Entnahmerate. Hier ergibt $800\,/\mathrm{s} \cdot 0{,}020\,\mathrm{s} = 16$ Nachrichten. Die diskrete Spitze von siebzehn liegt um eins darüber, weil der Get am Intervallende erst nach dem Put erfolgt. Bei dauerhaftem $\lambda > \mu$ wächst der Speicherbedarf mit der betrachteten Zeit unbegrenzt.

Für einen Burst, also eine zeitlich begrenzte Häufung, kann die Formel dagegen eine endliche Kapazität liefern. $(\lambda - \mu) \cdot T_b$ mit Burstdauer T_b ist dabei nur ein Überschlag für den Belegungszuwachs; Anfangsbelegung, Phasenlage und diskrete Zwischenstände müssen zusätzlich berücksichtigt werden. Enden im Beispiel alle Ankünfte nach Put zwanzig, liegt die Spitze vor dem gleichzeitigen Get bei siebzehn. Nach diesem Get bleiben sechzehn Nachrichten. Die weiteren Gets bei fünfundzwanzig bis einhundert Millisekunden leeren sie. Die letzte, bei zwanzig eingereihte Nachricht wird bei einhundert entnommen und wartet damit achtzig Millisekunden. Die Verarbeitung nach dem Get kommt für die Deadline noch hinzu. Speicherbedarf und Wartezeit müssen deshalb gemeinsam zur Anforderung passen.

Für Rückfragen über das Rechenmodell hinaus: In einer stationären, stabilen Warteschlange beschreibt Littles Gesetz die Beziehung $\overline{Q} = \lambda_{\text{eff}}\,\overline{W}$. Dabei ist $\overline{Q}$ die mittlere Queue-Belegung, λ_{eff} die tatsächlich angenommene Ankunftsrate und $\overline{W}$ die mittlere Wartezeit bis zur Entnahme. Diese Mittelwerte liefern allein keine maximale Wartezeit für einen Deadline-Nachweis. Network Calculus, ein mathematisches Verfahren zur Analyse begrenzter Datenflüsse, verwendet obere Ankunfts- und garantierte Bedienkurven; die größte vertikale Abweichung begrenzt den Rückstau. Bei einer anfangs leeren Queue, einer sogenannten Token-Bucket-Quelle mit höchstens $b + r\Delta$ Ankünften in jedem Intervall der Länge Δ und einer sofort verfügbaren konstanten Bedienrate $R \geq r$ beträgt die idealisierte Speicherschranke b. Dabei ist b die erlaubte Burstmenge, r die langfristige Zuflussrate und R die Bedienrate. Anlaufverzögerungen erfordern eine zusätzliche Betrachtung. Diese Vertiefung ist keine Voraussetzung für die Papieraufgaben.

Folie 20 / 50

Bei Überlast eine bewusste Entscheidung treffen

Regel	Geeignet, wenn …
Sender auf Platz warten lassen	die Quelle warten kann
neuen Datensatz verwerfen	Verlust erkannt und behandelt wird
älteren Zustand ersetzen	nur der neueste Stand benötigt wird
Fehler eskalieren	Verlust die Anforderung verletzt

Blockierter Softwarethread stoppt keine Hardwarequelle automatisch.

Notizen 20 / 50

Überlast bedeutet, dass die angebotene Arbeit die verfügbare Verarbeitung oder den Speicher überfordert. Eine Warteentscheidung beim Producer, dem Softwareerzeuger, erzeugt Backpressure, also Gegendruck zur Quelle. Das kann sinnvoll sein, wenn eine Berechnung später fortgesetzt werden darf. Eine unabhängig weiterlaufende Hardwarequelle wartet deshalb aber nicht automatisch. ADC bedeutet Analog-Digital-Wandler, UART Universal Asynchronous Receiver/Transmitter, serielle Schnittstelle, und DMA Direct Memory Access, direkter Hardware-Speicherzugriff. Diese Quellen brauchen einen eigenen Fluss- und Puffervertrag.
Neue Datensätze zu verwerfen bewahrt die ältere Historie, hinterlässt jedoch Lücken. Ältere Zustände zu ersetzen priorisiert Aktualität und ist eine andere Semantik als FIFO, First In First Out. Eskalation bedeutet eine sichtbare Behandlung, etwa Abbruch eines Versuchs oder eine dokumentierte Fehleranzeige. Ein still erhöhtes Diagnosefeld allein erfüllt noch keine Sicherheits- oder Qualitätsanforderung. Papieraufgabe 2 verbindet die Lastrechnung mit der Auswahl einer solchen Regel, damit die Rechenzahl zu einer begründeten Entwurfsentscheidung führt.

Folie 21 / 50

Threadsichere Queue, trotzdem Anwendungsdeadlock

1. Producer hält Mutex A.
2. Volle Queue: Producer wartet im Put auf Platz.
3. Consumer braucht A, bevor er das nächste Get ausführt.

Producer wartet auf Consumer; Consumer wartet auf Producer.
Den Mutex vor dem blockierenden Put freigeben, wenn der Vertrag es erlaubt.

Notizen 21 / 50

Producer ist der Nachrichtenerzeuger, Consumer der Empfänger, Put das Einreihen und Get das Entnehmen. Ein Mutex schützt eine Ressource und hält einen Besitzer fest. Im Beispiel besitzt der Producer Mutex A und versucht anschließend einen wartenden Put auf eine volle Queue. Er kann den Mutex erst nach erfolgreichem Put freigeben. Der Consumer versucht vor seiner nächsten Entnahme ebenfalls A zu erwerben. Weil A belegt ist, kann er nicht zum Get gelangen.
Damit entsteht ein Deadlock, eine Verklemmung durch einen geschlossenen Wartezyklus. Die Queue kann ihre internen Strukturen völlig korrekt schützen und trotzdem diesen Anwendungsfehler nicht verhindern. Auch die Prioritätsvererbung des Mutex erzeugt keinen freien Queue-Platz, wenn der Besitzer selbst blockiert ist. Ein geeigneter Entwurf kopiert die benötigten Daten im geschützten Abschnitt, gibt den Mutex frei und führt dann den möglicherweise blockierenden Put mit der privaten Kopie aus. Das gilt nur, wenn Inhalt und Lebensdauer der Kopie den Anwendungsvertrag erfüllen. Die bekannte Ressourcenanalyse aus Vorlesung 4 bleibt somit auch beim Datentransport notwendig.

Folie 22 / 50

Wertkopie entkoppelt lokale Nachrichten

```
SensorMsg msg = { .sample = 10u, .seq = 1u };
if (osMessageQueuePut(q, &msg, 0u, 0u) == osOK) {
    msg.sample = 99u;
}
```

Der Consumer erhält weiterhin `sample = 10`, sofern niemand die Queue verändert. Queue-Slots speichern die festgelegte Elementgröße.

Notizen 22 / 50

Kernaussage: Nach einem erfolgreichen Put sind die lokale Variable und die gespeicherte Nachricht zwei unabhängige Objekte. Spätere Änderungen am Original erreichen die Queue nicht.
Der C-Code zeigt eine kleine Wertnachricht. `sample` ist der Nutzwert, `seq` die Sequenznummer und `q` ein gültiger Handle, also Bezeichner einer Queue mit der Elementgröße `sizeof(SensorMsg)`. Put bedeutet Einreihen; `osOK` bestätigt den Erfolg. Nach dem erfolgreichen Put wird `msg.sample` auf neunundneunzig gesetzt, ein späterer erfolgreicher Get, also Entnahmeaufruf, liefert trotzdem zehn. Das entspricht den zwei Kopiervorgängen aus Folie 7: Put kopiert vom Producer, dem Erzeuger, in den Queue-Speicher; Get kopiert von dort zum Consumer, dem Empfänger. Die lokale Nachricht darf nach dem erfolgreichen Put wiederverwendet werden.
Der Zusatz „sofern niemand die Queue verändert“ grenzt den Ablauf ein. Ein zweiter Consumer könnte die Nachricht zuvor entnehmen, oder `osMessageQueueReset`, eine Verwaltungsfunktion zum Leeren der Queue, könnte sie verwerfen. Betrachtet wird hier eine weiterhin gespeicherte Nachricht.
Für die folgenden Folien ist entscheidend: Die Kopie ist **flach**. Der Kernel kopiert die festgelegte Anzahl von Bytes und interpretiert deren Inhalt nicht. Der Unterschied zwischen einem eingebetteten Array und einem Pointerfeld wird an zwei C-Strukturtypen deutlich:

```
typedef struct { uint8_t data[8]; } MsgA;
typedef struct { uint8_t *data;   } MsgB;
```

`uint8_t` bezeichnet einen vorzeichenlosen Acht-Bit-Typ. Bei `MsgA` gehört das gesamte Array zum Strukturinhalt; seine acht Nutzdatenbytes werden mitkopiert. Bei `MsgB` gehört nur der Pointer, also die Adresse, zur Struktur. Die referenzierten Nutzdaten werden weder dupliziert noch gegen gleichzeitige Änderungen geschützt. Das Feld lässt sich in beiden Fällen mit `msg.data[i]` ansprechen, wobei `i` ein gültiger Index sein muss. Bei `MsgB` muss der Pointer zusätzlich auf einen gültigen, ausreichend großen Puffer zeigen. Die gleiche Zugriffsschreibweise bedeutet somit unterschiedliche Speicherverhältnisse. Kleine reine Wertnachrichten benötigen während des Put-Aufrufs eine gültige Quelle; Pointer-Nachrichten brauchen darüber hinaus einen Besitzvertrag für die Lebensdauer des referenzierten Puffers.

Folie 23 / 50

Eine Pointerkopie kopiert kein Bild

```
uint8_t *ptr = image_buffer;
/* q_ptr wurde mit sizeof(ptr) angelegt. */
osStatus_t st = osMessageQueuePut(q_ptr, &ptr, 0u, 0u);
```

- Kopiert wird die Adresse, nicht der Bildinhalt
- Puffer muss gültig und unverändert verfügbar bleiben
- `static` verlängert die Lebensdauer, schützt aber nicht vor Überschreiben

Notizen 23 / 50

Kernaussage: Wird ein Pointer eingereiht, transportiert die Queue nur eine Adresse. Die Nutzdaten bleiben, wo sie sind, und ihre Gültigkeit ist Sache der Anwendung.

Der C-Ausschnitt ist Lehrcode. `uint8_t` bezeichnet einen vorzeichenlosen Acht-Bit-Typ, `image_buffer` einen separat bereitgestellten Bildpuffer und `ptr` dessen Adresse. `q_ptr` ist der Handle, also Bezeichner einer Queue für Pointerwerte. Im betrachteten STM32H755-Modell ist ein solcher Pointer vier Byte groß. `st` ist der Rückgabestatus; erst `osOK` bestätigt eine erfolgreiche Übergabe. Zwei Fehlerquellen verdienen besondere Aufmerksamkeit.

Erstens: `&ptr` statt `ptr`. Put erwartet die Adresse des zu kopierenden Objekts. Dieses Objekt ist hier die Pointervariable; deshalb ist `&ptr` richtig. Wird bei einer Elementgröße von vier Byte versehentlich `ptr` übergeben, kopiert der Kernel vier Byte vom Anfang des Bildpuffers. Bei einem Graustufenbild mit einem Byte pro Pixel wären das vier Pixelwerte. Der Consumer erhält diese Bytes als vermeintlichen Pointer. Beim späteren Zugriff auf diese falsche Adresse drohen falsche Daten oder ein HardFault, eine Prozessor-Fehlerausnahme. Der Parameter von `osMessageQueuePut` hat den allgemeinen Zeigertyp `const void *`; beide Argumentformen sind in C zulässig, deshalb erkennt der Compiler diesen Vertragsfehler üblicherweise nicht.

Beim Get, dem Entnahmeaufruf, gilt die Regel spiegelbildlich. Die Zieladresse ist die Adresse der Pointervariable des Consumers:

```
uint8_t *rx = NULL;
osStatus_t rx_status = osMessageQueueGet(
    q_ptr, &rx, NULL, 0u);
```

`osStatus_t` ist der Statusdatentyp. Nur bei `rx_status == osOK` liegt ein neu empfangener Pointer vor, dessen Puffer gemäß Besitzvertrag benutzt werden darf. `NULL` fordert keine Rückgabe einer Nachrichtenpriorität an; Timeout null bedeutet hier sofortige Rückkehr, falls die Queue leer ist. Ein versehentliches `rx` statt `&rx` würde die Nutzdaten an eine falsche Zieladresse schreiben oder einen Parameterfehler auslösen.

Zweitens: die Elementgröße bei der Anlage. Für `uint8_t image_buffer[76800]` und eine Queue mit zehn Plätzen ergeben sich im STM32H755-Modell folgende Größen:

Ausdruck	Byte pro Element	Wirkung
`sizeof(ptr)`	4	Richtig für einen Pointer pro Queue-Platz.

Notizen, Fortsetzung 23 / 50

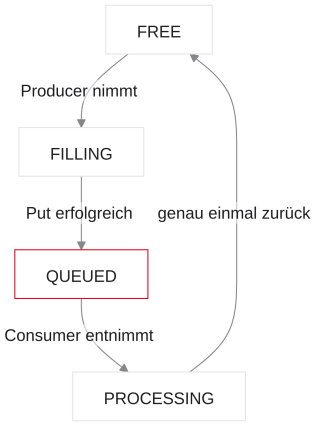
`sizeof(*ptr)`	1	Nur ein Byte des Pointerobjekts wird übertragen; die Adresse ist unvollständig.
`sizeof(image_buffer)`	76 800	Platz für eine vollständige Bildkopie je Element; zehn Plätze benötigen 750 KiB reinen Nutzspeicher.

`sizeof` ermittelt eine Größe in Byte, ohne die Bilddaten zu lesen. Der letzte Tabellenfall ist nur dann eine Bild-Wertkopie, wenn auch der Bildpuffer selbst als Put-Quelle übergeben wird. Mit unverändertem `&ptr` würde der Kernel weit über die Pointervariable hinaus lesen. Die große Queue-Anlage kann bei unzureichendem verfügbarem Speicher scheitern; ein `NULL`-Handle muss den Initialisierungsablauf stoppen. KiB bedeutet Kibibyte mit 1024 Byte. Der zusätzliche Verwaltungsbedarf und die tatsächlich nutzbaren Speicherregionen sind gesondert zu prüfen. Die vier Byte Pointergröße sind eine Zielplattformannahme und gelten beispielsweise auf einem 64-Bit-PC nicht allgemein. Für die Pointer-Queue ist `sizeof(uint8_t *)` oder ein eigener Nachrichtentyp eine klare Wahl.
Nach erfolgreicher Pointerübergabe muss der Bildpuffer weiter existieren und bis zum Ende der Consumer-Nutzung vor Überschreiben geschützt sein. Ein lokales Array einer beendeten Funktion ist dafür ungeeignet. `static` verlängert die Lebensdauer, schützt aber nicht vor erneutem Befüllen; dasselbe gilt gegenüber einer laufenden Hardwareübertragung. Der Ausdruck Copy by Reference bezeichnet hier diesen Besitzvertrag. Technisch bleibt die Queue-Operation eine Wertkopie des Pointers.

Folie 24 / 50

Pufferbesitz im Kreislauf

Gescheiterter Put: Verantwortung bleibt beim Producer.



Notizen 24 / 50

Die Abbildung beschreibt Zustände eines Datenpuffers, keine Threadzustände. Deshalb zeigt der Zustandskreislauf den Besitz und keine Präemption. FREE bedeutet verfügbar, FILLING vom Producer befüllt, QUEUED erfolgreich zur Consumerseite übergeben und PROCESSING vom Consumer verarbeitet. Die

Notizen, Fortsetzung 24 / 50

Pfeile werden von oben nach unten und dann zurück zum Anfang gelesen. Die rote Markierung hebt die erfolgreiche Übergabe hervor. Producer ist der Erzeuger, Consumer der Verbraucher und Put das Einreihen einer Adresse oder eines Deskriptors.
Eine Free-Pool-Struktur enthält verfügbare Pufferidentitäten; die Ready-Queue enthält übergebene Puffer. Bei erfolgreichem Put darf der Producer den Puffer nicht mehr verändern. Der Consumer gibt ihn nach abgeschlossener Nutzung genau einmal zurück. Bei fehlgeschlagenem Put bleibt der Producer verantwortlich und muss den Puffer behalten, später erneut übergeben oder einmal zurückgeben. Er darf nicht zugleich erneut einreihen und zurückgeben. Die tatsächliche Ausführung kann einen geweckten Consumer schon vor Rückkehr des Producer-Aufrufs starten lassen. Der Besitzvertrag muss darum am erfolgreichen Übergabepunkt gelten, nicht erst bei einer später geschriebenen Diagnosezeile.

Folie 25 / 50

Queue-Speicher und Payload-Pool getrennt zählen

Lehrannahmen: zehn Plätze, 4-Byte-Pointer, Bilder mit 75 KiB.

Anteil	Nutzspeicher
zehn reine Pointer-Slots	40 Byte
zehn Bildpuffer	750 KiB
nur zwei Bildpuffer	150 KiB, höchstens zwei Bilder gleichzeitig

Kontrollblock, Alignment und Allokator kommen hinzu.

Notizen 25 / 50

Kernaussage: Queue-Plätze und Bildpuffer sind getrennte Ressourcen. Bei zehn Queue-Plätzen und nur zwei Puffern bestimmt die Pufferzahl, wie viele unabhängige Bilder gleichzeitig unterwegs sein können.
Payload bedeutet Nutzdaten; ein Payload-Pool ist ein Vorrat separat bereitgestellter Datenpuffer. KiB steht für Kibibyte mit 1024 Byte. Das Lehrmodell nimmt zehn Queue-Plätze, Pointer mit vier Byte und Bilder mit je fünfundsiebzig KiB an. Zehn reine Pointer-Slots benötigen $10 \cdot 4 = 40$ Byte Elementnutzspeicher. Zehn Bildpuffer benötigen $10 \cdot 75 = 750$ KiB, zwei dagegen 150 KiB. Diese Zahlen beschreiben verschiedene Speicherbereiche: Eine Queue mit zehn Pointerplätzen erzeugt keine zehn Bildpuffer.
Ein anschauliches Bildformat ist QVGA, Quarter Video Graphics Array, hier mit 320×240 Bildpunkten. Bei einem Byte Grauwert pro Pixel hat ein Bild genau $320 \cdot 240 \cdot 1 = 76\,800$ Byte, also $76\,800/1024 = 75$ KiB. Das Bildformat veranschaulicht die angenommene Nutzdatenmenge.
Die entscheidende Tabellenzeile ist die dritte. Der Besitzvertrag aus der vorherigen Folie setzt voraus, dass jeder Puffer eindeutig verwaltet wird und höchstens einmal zur Consumerseite unterwegs ist. Mit nur zwei Bildpuffern enthält die Queue deshalb höchstens zwei verschiedene Bildadressen, obwohl sie zehn Plätze besitzt. Mindestens acht Queue-Plätze bleiben ungenutzt. Meist warten weniger Adressen: Ein Puffer kann gerade vom Producer, dem Erzeuger, befüllt oder vom Consumer, dem Verbraucher, verarbeitet werden. Diese Puffer belegen keinen Queue-Platz, gehören aber weiterhin zum begrenzten Pool. Die Queue-Länge begrenzt wartende Adressen; die Pufferzahl begrenzt unabhängige Bilder über den gesamten Besitzkreislauf.

Notizen, Fortsetzung 25 / 50

Daraus folgen drei gebräuchliche Anordnungen für vollständige Bildpuffer:

- **Ein Puffer:** Producer und Consumer nutzen ihn abwechselnd. Das nächste vollständige Bild darf erst entstehen, wenn die Verarbeitung des vorherigen beendet ist.
- **Zwei Puffer, Double Buffering:** Ein Puffer kann befüllt werden, während der andere verarbeitet wird. Die beiden Arbeitsschritte können sich an unterschiedlichen Bildern überlappen.
- **Drei Puffer, Triple Buffering:** Zusätzlich zu einem Puffer beim Producer und einem beim Consumer kann ein vollständiges Bild in der Queue warten. Das kann begrenzte Laufzeitschwankungen auffangen, bevor eine Seite auf einen freien Puffer warten muss.

Auf dem einzelnen M4-Prozessorkern werden die Threadanweisungen weiterhin nacheinander ausgeführt. Echte gleichzeitige Arbeit kann beispielsweise entstehen, wenn eine Hardwareübertragung einen Puffer füllt, während der Kern den anderen verarbeitet. Zusätzliche Puffer erlauben zeitliche Entkopplung und können zugleich die Wartezeit erhöhen. Eine dauerhaft langsamere Verarbeitung wird durch Double oder Triple Buffering nicht schneller.
Kontrollblock, Alignment, also Speicherausrichtung, und Allokatorverwaltung erhöhen den Gesamtbedarf. Ein Deskriptor mit Pointer, Länge und Sequenznummer ist außerdem größer als ein reiner Pointer. Die Zahlen sind eine Modellrechnung über Nutzspeicher. Ob ein Entwurf in den verfügbaren RAM, Random Access Memory oder Arbeitsspeicher, passt, muss anhand der Linker-Map, der Speicherbelegung des gebundenen Programms, der erreichbaren Speicherregionen und der konkreten Adapteranforderungen geprüft werden.

Folie 26 / 50

Eine Queue der Länge eins ist noch keine Mailbox

Start leer, Sender bietet 10, 20, 30; kein Leser dazwischen.

Vertrag	Späterer Inhalt
FIFO mit drei Plätzen	10, 20, 30
Länge 1, Put mit Timeout 0	10; zwei Ablehnungen
Latest-Value mit Overwrite	30

„Mailbox“ meint hier ausdrücklich den jüngsten Zustand.

Notizen 26 / 50

FIFO heißt First In First Out und bewahrt erfolgreich eingereihte Nachrichten in ihrer Reihenfolge. Latest-Value bedeutet jüngster Wert. Overwrite ist ein ausdrücklich erlaubtes Ersetzen eines noch ungelesenen älteren Zustands. Der Begriff Mailbox wird von Betriebssystemen unterschiedlich verwendet; in dieser Vorlesung bezeichnet er diesen Latest-Value-Vertrag.
Die Länge-1-Queue nimmt den ersten Wert zehn auf. Beide späteren Puts mit Timeout null werden bei Vollbelegung abgelehnt und lassen zehn stehen. Timeout null ist deshalb keine Näherung an Overwrite. Eine echte Overwrite-Operation ersetzt dagegen zunächst zehn durch zwanzig und danach durch dreißig. Im nativen FreeRTOS existiert xQueueOverwrite für eine Queue der Länge eins. CMSIS-RTOS2 bietet hier

Notizen, Fortsetzung 26 / 50

keine entsprechende standardisierte Queue-Overwrite-Funktion. Ein CMSIS-Handle wird nicht zu einem nativen Handle gecastet. Auch Get gefolgt von Put bildet gegenüber anderen Teilnehmern kein atomares Ersetzen. Wer beim Display nur Aktualität braucht, muss somit einen passenden Vertrag implementieren, statt lediglich die Kapazität auf eins zu setzen.

Folie 27 / 50

Aktuellen Zustand als geschützten Snapshot lesen

Ein Thread veröffentlicht Wert, Version und Zeitpunkt unter einem Mutex.

- Leser kopiert denselben Datensatz unter demselben Mutex
- Neue Version zeigt eine Änderung
- Zwischenversionen dürfen verloren gehen
- Benachrichtigung folgt auf Veröffentlichung
- ISR verwendet keinen Mutex

Notizen 27 / 50

Kernaussage: Für einen aktuellen Zustand genügt ein gemeinsam geschützter Datensatz; eine Queue ist dafür nicht erforderlich. Der Mutex sorgt dafür, dass alle Felder zusammenpassen. Die Versionsnummer zeigt, ob sich seit dem letzten Lesen etwas geändert hat. Eine Historie entsteht dabei bewusst nicht.
Diese Folie zeigt eine konkrete Umsetzung des Latest-Value-Vertrags zwischen Threads: Gelesen wird der zuletzt veröffentlichte, zusammenhängende Zustand. Sie schließt an Folie 26 an, die die Grenzen einer Queue der Länge eins beim gewöhnlichen Put gezeigt hat, und arbeitet den „Einzelwert mit Mutex“ aus Folie 5 zu einem Datensatz mit mehreren Feldern aus.
Ein Snapshot ist eine zusammengehörige Momentaufnahme mehrerer Felder. Ein veröffentlichender Thread schreibt beispielsweise Wert, Versionsnummer und Zeitstempel in einem kurzen, durch Mutex geschützten Abschnitt. Ein Mutex ist eine Sperre mit Besitzer. Alle beteiligten Leser kopieren unter derselben Sperre den vollständigen Datensatz und geben die Sperre danach frei. So stimmen die Felder zusammen, sofern kein weiterer ungeschützter Teilnehmer schreibt.
Die Version kennzeichnet Änderungen. Sie beweist keine Zustellung aller Zwischenversionen. Ein Leser kann etwa direkt von Version eins auf fünf springen, wenn er später liest. Für einen aktuellen Zustand kann das erlaubt sein. Wenn eine Benachrichtigung verwendet wird, folgt sie auf die vollständige Veröffentlichung, damit der geweckte Leser nicht erst einen alten Stand findet. ISR steht für Interrupt Service Routine, Interruptbehandler. Er darf keinen CMSIS-Mutex erwerben oder freigeben. Bei hardwareerfassten Daten übergibt daher ein erlaubter kurzer Kanal die Rohdaten an den veröffentlichenden Thread. Diese Lösung baut auf Vorlesung 4 auf und bleibt heute ein Entwurfsmuster, kein zusätzliches Pflichtprogramm.

Folie 28 / 50

Mehrere Consumer: Arbeitsverteilung oder Zustellung?

- Ein Get entnimmt eine Nachricht für einen Consumer
- Zwei Get-Consumer bekommen nicht beide dieselbe Nachricht
- Gemeinsamer Snapshot: mehrere Leser, möglicherweise verschiedene Versionen
- Jeder Pflichtempfänger braucht einen eigenen Zustellvertrag

Beispiel: zwei eigene Queues, jeweils mit Überlaufregel.

Notizen 28 / 50

Consumer bezeichnet einen Nachrichtenverbraucher und Get die Entnahme. Bei zwei Threads an derselben Queue wird jede Nachricht einmal entnommen. Welcher Consumer sie erhält, hängt vom Ablauf und den Kernel-Wartelisten ab. Das ist Arbeitsverteilung, etwa zwei Bearbeiter einer Auftragsfolge. Es ist kein Broadcast. Broadcast bedeutet Zustellung einer Nachricht an alle vorgesehenen Empfänger.
Ein Snapshot, eine konsistente gemeinsame Momentaufnahme, kann von mehreren Lesern kopiert werden. Sie lesen zu verschiedenen Zeitpunkten möglicherweise unterschiedliche Versionen. Gemeinsame Lesbarkeit bedeutet deshalb ebenfalls nicht, dass jeder alle Ereignisse bekommt. Für zwei Pflichtempfänger kann der Sender zwei getrennte Queues benutzen. Dann muss er aber jeden Put-Status getrennt behandeln. Erfolg an Queue A und Überlauf an Queue B wäre eine teilweise Zustellung, kein gemeinsamer Erfolg. Dieses Beispiel führt keinen vollständigen Bestätigungsmechanismus ein, sondern macht die Anforderung sichtbar. Bei der Papieraufgabe zu Historie und Zustand ist immer zuerst zu klären, ob Daten einmal bearbeitet, mehrfach gelesen oder zuverlässig an jeden Empfänger geliefert werden müssen.

Folie 29 / 50

Papierprüfung: die passende Aussage begründen

1. Head = Tail: genügt das als Leerbeweis?
2. Erfolgreicher Put: ist die Nachricht schon verarbeitet?
3. Zehn Pointerplätze: existieren damit zehn Bilder?
4. Zustand 30 gewünscht: liefert eine volle 1er-Try-Put-Queue ihn?

Antwort jeweils mit Voraussetzung oder Gegenbeispiel.

Notizen 29 / 50

Diese Verständnisfragen verbinden die erste Hälfte. Head und Tail sind die nächsten Entnahme- und Einfügepositionen des Ringmodells. Sie können bei leerem und vollem Ring gleich sein; Count, die Belegung, entscheidet. Ein Put ist das Einreihen. Sein Erfolgsstatus bestätigt die Kopie, aber keine abgeschlossene Verarbeitung. Der Consumer, der Empfänger, kann noch Blocked oder Ready gewesen sein und muss erst ausgewählt werden.
Zehn Pointerplätze sind zehn speicherbare Adressen. Die tatsächlichen Bildpuffer werden separat angelegt und gezählt. Mit nur zwei Puffern existieren weiterhin nur zwei unabhängige Bilder. Try-Put meint Einfügen

Notizen, Fortsetzung 29 / 50

ohne Warten. Auf einer vollen Queue der Länge eins lehnt es den neuen Wert ab; der alte Wert bleibt. Nur ein expliziter Latest-Value-Vertrag, also Ersetzen durch den jüngsten Zustand, könnte dreißig liefern. Die begründeten Antworten sind wichtiger als vier Wörter wahr oder falsch. Dieselben Trennungen gelten im zweiten Teil, wenn ein Hardwareereignis statt eines Softwarethreads Daten erzeugt. Hinzu kommen dann die Aufrufregeln des Interruptkontexts.

Folie 30 / 50

Ein Interrupt ist kein zusätzlicher Thread

- IRQ unterbricht die Ausführung auf diesem Kern
- ISR erledigt die Interruptbehandlung
- Ein höher dringlicher Interrupt kann die ISR unterbrechen
- ISR darf nicht wie ein Thread auf Daten oder Platz blockieren
- Zweiter Kern und Hardware können weiterarbeiten

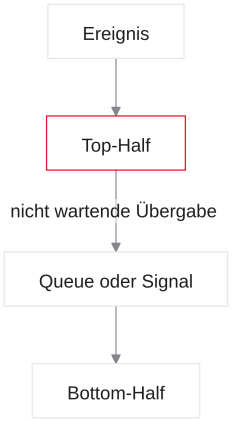
Notizen 30 / 50

IRQ steht für Interrupt Request, Interruptanforderung. ISR steht für Interrupt Service Routine, Interruptbehandler. Ein Hardwareereignis kann den laufenden Thread auf demselben Kern unterbrechen und den Behandler ausführen lassen. Der Behandler ist dabei kein weiterer Thread mit eigenem wartenden Betriebssystemkontext. Die Threads werden durch den Interrupt nicht einfach alle auf den Zustand Blocked gesetzt.
Eine höher dringliche Interruptquelle kann je nach Prioritäten und Maskierung auch eine ISR unterbrechen. Ein Thread mit hoher Betriebssystempriorität verdrängt dagegen keine noch aktive Hardware-ISR. Der zweite Kern und DMA, Direct Memory Access oder direkter Hardware-Speicherzugriff, können unabhängig weiterarbeiten. Eine lange ISR verzögert Threadfortschritt und möglicherweise andere Interruptbehandlungen. Daraus folgt die Beschränkung auf die sofort notwendige Arbeit. Der Behandler kann nicht auf eine volle Queue warten, bis ein Thread endlich einen Platz entnimmt: Dieser Thread müsste auf demselben Kern erst wieder laufen können. Das aufgeschobene Verarbeitungsmuster trennt diese beiden Verantwortlichkeiten.

Folie 31 / 50

Top-Half sichert, Bottom-Half verarbeitet

Deferred Interrupt Processing: aufschiebbare Arbeit in einen Thread verlagern.



Notizen 31 / 50

Deferred Interrupt Processing bedeutet aufgeschobene Interruptverarbeitung. Die Top-Half, obere Hälfte, ist der unmittelbar ausgeführte Behandler. Die Bottom-Half, untere Hälfte, ist ein Worker, also ein verarbeitender Thread. Die Abbildung wird von oben nach unten gelesen. Der rote Rahmen hebt die sofort notwendige Hardwarearbeit hervor. Die Pfeile beschreiben Daten- und Signalübergabe; die Zeichnung ist kein zeitlich maßstäbliches Schedulingdiagramm.
Die Top-Half quittiert das Ereignis nach dem Gerätevertrag und sichert die Daten, die sonst verloren gehen könnten. Quittieren bedeutet, die erkannte Anforderung im vorgesehenen Hardwarepfad zu bestätigen. Die Reihenfolge von Datenlesen und Flagbehandlung ist gerätespezifisch. Die nicht wartende Übergabe speichert entweder kleine Rohdaten in einer Queue oder meldet vorhandene Arbeit. Die Bottom-Half führt Berechnungen und erlaubtes Warten aus. Deren Priorität folgt den Fristen und der Last; sie ist nicht automatisch maximal. Das Auslagern verkürzt den unmittelbar belegten Interruptpfad, beseitigt aber weder den Kapazitätsbedarf noch die Gesamtzeit bis zum Ergebnis.

Folie 32 / 50

Was in der ISR sofort nötig ist

- Bedrohte Rohdaten sichern
- Hardwareflag nach Gerätevertrag behandeln
- Kleinen Datensatz oder Arbeitshinweis übergeben
- Status prüfen und knappe Diagnose führen

Keine lange Rechnung, kein `printf`, kein Mutex, kein blockierendes Warten.

Notizen 32 / 50

ISR bedeutet Interrupt Service Routine, Interruptbehandler. Rohdaten sind hier der noch nicht vollständig

Notizen, Fortsetzung 32 / 50

ausgewertete Eingang, etwa ein Registerwert oder die Kennzeichnung eines abgeschlossenen Puffers. Würde die Hardware ein Register vor dem Workerstart überschreiben, reicht bloßes Aufwecken nicht aus. Die Daten müssen rechtzeitig kopiert oder in einem gültigen Hardwarepuffer erhalten werden.
printf ist die formatierte C-Ausgabe. Sie kann umfangreiche Bibliotheks-, Geräte- und Synchronisationsarbeit auslösen und ist deshalb kein geeigneter kurzer Diagnosepfad im Behandler. Ein Mutex ist eine Besitzsperre; seine CMSIS-Aufrufe sind im Interruptkontext unzulässig. Blockierendes Warten kann den Thread, der Fortschritt ermöglichen müsste, am Laufen hindern. Eine kurze Übergabe bleibt trotzdem endlich: Der Rückgabestatus kann fehlenden Platz melden. Diagnosezähler brauchen eindeutige Schreiber. Ein volatile-Zähler, den ISR und Thread beide mit ++ ändern, wird durch volatile nicht nebenläufigkeitssicher. Falsche API-Nutzung kann zu Fehlerstatus, Assertion, Inkonsistenz oder Stillstand führen; ein bestimmter Hardware-Fault ist kein universell garantierter Ausgang.

Folie 33 / 50

Token oder Nachricht: was soll erhalten bleiben?

Bedarf	Passender Kanal
„Arbeit ist vorhanden“	leerer binärer Semaphor
jedes erfasste Ereignis mit Daten	Queue, Timeout 0, Überlaufregel
jüngster Zustand	expliziter Snapshot- oder Overwrite-Vertrag

Drei Releases ohne Acquire, Maximum 1: ein gespeichertes Token.

Notizen 33 / 50

Ein Semaphor ist ein begrenzter Zähler von Berechtigungen oder Hinweisen. Ein binärer Semaphor erlaubt höchstens ein Token und beginnt für Ereignissignalisierung leer. Release stellt ein Token bereit, Acquire verbraucht es. Bei drei Releases ohne dazwischenliegendes Acquire gelingt im anfangs leeren Semaphor mit Maximum eins nur der erste; die anderen melden fehlende Kapazität. Ein späterer Worker kann deshalb höchstens einen gespeicherten Hinweis erhalten.
Eine Queue ist eine begrenzte Nachrichtenwarteschlange. Ein Put mit Timeout null kann kleine Daten aus einer erlaubten ISR übernehmen, meldet bei Vollbelegung aber ebenfalls einen Fehler. ISR heißt Interrupt Service Routine, Interruptbehandler. „Jedes Ereignis“ beschreibt die Anforderung, nicht die unbegrenzte Leistungsfähigkeit des Kanals. Ein Snapshot ist eine konsistente Momentaufnahme, Overwrite ein ausdrücklich erlaubtes Ersetzen. Diese Mittel eignen sich für den jüngsten Zustand, brauchen jedoch weiterhin einen sicheren Veröffentlichungsweg. Ein Token enthält keine Messbytes. Wenn der Worker nur geweckt wird und das ursprüngliche Register inzwischen einen neuen Wert trägt, ist die frühere Messung bereits verloren.

Folie 34 / 50

Drei Bedingungen für einen ISR-Aufruf

1. Funktion im Interruptkontext zugelassen?
2. Parameter ohne blockierendes Warten?
3. Interruptpriorität für Kernelaufrufe zulässig?

CMSIS-RTOS2	ISR-Regel
Queue Put/Get, Semaphor Acquire	nur Timeout 0
Semaphor Release	erlaubt bei passendem Kontext
Mutex Acquire/Release	unzulässig

Notizen 34 / 50

ISR steht für Interrupt Service Routine, Interruptbehandler. CMSIS-RTOS2 ist die Betriebssystem-Schnittstelle des Common Microcontroller Software Interface Standard in Version zwei. Die Tabelle nennt deren grundlegende Kontextverträge. Queue Put und Get bedeuten Einreihen und Entnehmen. Semaphor Acquire und Release verbrauchen und erzeugen ein Token. Ein Mutex ist eine Besitzsperre und darf über diese Schnittstelle nicht aus dem Behandler erworben oder freigegeben werden.
Alle drei Fragen müssen positiv beantwortet werden. Timeout null erfüllt nur die zweite. So bleibt osMutexAcquire mit null trotzdem verboten. Die Interruptpriorität ist eine zusätzliche Anforderung des konkreten FreeRTOS-Ports. Ein erlaubter Name und nicht wartende Parameter helfen nicht, wenn der aufrufende Interrupt außerhalb dieses Bereichs liegt. Auch das Anlegen, Zurücksetzen oder Löschen einer Queue gehört nicht in diesen Callback. Die Tabelle ist keine vollständige Liste aller Funktionen. Prüfen Sie bei weiteren APIs deren dokumentierten Kontext, anstatt aus einem ähnlichen Namen automatisch auf Zulässigkeit zu schließen. Nun müssen wir die Hardwarepriorität von der bereits bekannten Threadpriorität trennen.

Folie 35 / 50

Zwei Prioritätsskalen mit verschiedener Richtung

Skala	Bedeutung
CMSIS-Threadpriorität	höherer Wert: höherer Threadvorrang
logische NVIC-Priorität	kleinere Zahl: höhere IRQ-Dringlichkeit

Ein Worker mit `osPriorityHigh` repariert keine unzulässige IRQ-Priorität.
Kernwissen: Threadvorrang und Interruptdringlichkeit trennen.

Notizen 35 / 50

CMSIS bedeutet Common Microcontroller Software Interface Standard; osPriorityHigh ist eine hohe Threadpriorität seiner Betriebssystem-Schnittstelle. Threadpriorität bestimmt, welche ausführungsbereite Aufgabe der Scheduler auswählt. NVIC steht für Nested Vectored Interrupt Controller, den verschachtelten Interruptcontroller des Cortex-M. IRQ bedeutet Interrupt Request, Interruptanforderung. Die logische NVIC-Priorität hat die umgekehrte Zahlenrichtung: Null ist besonders dringlich, größere Zahlen weniger dringlich.
FreeRTOS schützt bestimmte Kerneloperationen durch Maskierung eines Interruptbereichs. Interrupts, die darüber besonders dringlich bleiben, dürfen diese Kerneloperationen nicht selbst aufrufen. Ein Worker kann beliebig hoch unter den Threads stehen und diese Hardwaregrenze trotzdem nicht verändern. Für das Verständnis müssen Sie beide Skalen unterscheiden und die drei Bedingungen des vorherigen Abschnitts gemeinsam prüfen: erlaubte Funktion, kein blockierendes Warten und zulässige Interruptpriorität. Die genaue Zahl der implementierten Bits und ihre Gruppierung werden am konkreten Projekt geprüft. Die folgende Folie vertieft deren Registerdarstellung an ausdrücklich vorgegebenen Zahlen; daraus entsteht keine universelle Einstellung für jedes Board.

Folie 36 / 50

Vertiefung: Prioritätswert kodieren

Vier Bits, passende Gruppierung, logische Grenze fünf:

Logischer NVIC-Wert	Kernel-API aus IRQ
0–4	unzulässig
5–15	erlaubte Funktionen, ohne Warten

$$5 \cdot 2^{8-4} = 80 = 0\text{x}50$$

HAL-Setter erhält 5, nicht den Registerwert 0x50.

Notizen 36 / 50

Diese Implementierungsvertiefung erklärt den Unterschied zwischen einer logischen Priorität und ihrer Registerkodierung. NVIC ist der Nested Vectored Interrupt Controller, der Interruptcontroller. IRQ bedeutet Interrupt Request, API Application Programming Interface, Programmierschnittstelle, und HAL Hardware Abstraction Layer, Hardwareabstraktion. CMSIS bedeutet Common Microcontroller Software Interface Standard. Im Lehrbeispiel sind vier Prioritätsbits implementiert und gemäß den FreeRTOS-Portforderungen gruppiert. Die logische Kernelaufrufgrenze beträgt fünf. Werte null bis vier sind zu dringlich für die betrachteten Kernelaufrufe; fünf bis fünfzehn dürfen nur die dort erlaubten nicht wartenden Funktionen benutzen.
Im Acht-Bit-Prioritätsregister werden die vier implementierten Bits nach oben ausgerichtet. Deshalb wird fünf mit zwei hoch vier multipliziert: achtzig dezimal oder 0x50 hexadezimal. configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY bezeichnet in üblichen Cube-Konfigurationen die unverschobene Grenze; configMAX_SYSCALL_INTERRUPT_PRIORITY den Registerwert. Der HAL-

Notizen, Fortsetzung 36 / 50

Prioritätssetter erhält dagegen die logische fünf. Mit vorgegebener Bitzahl lässt sich die Umrechnung auf Papier nachvollziehen. Bei der eigenen Integration müssen Definitionen und Gruppierung des tatsächlichen Projekts geprüft werden. Die kleine CMSIS-Queue-Aufgabe benötigt danach eine gültige konfigurierte Priorität, keine selbst geschriebene Registerzugriffsroutine.

Folie 37 / 50

Vertiefung: der CMSIS-Adapter

`osMessageQueuePut` aus Thread oder erlaubter ISR:

- Adapter erkennt den Aufrufkontext
- Threadpfad: wartende oder sofortige Queue-Funktion
- ISR-Pfad: native `FromISR`-Funktion, Timeout 0
- Adapter fordert den erforderlichen Kontextwechsel an

Im CMSIS-Labor übernimmt der Adapter die Wechselanforderung.

Notizen 37 / 50

Diese Folie erläutert die Implementierung hinter der verwendeten Schnittstelle. CMSIS-RTOS2 ist die zweite Betriebssystem-Schnittstelle des Common Microcontroller Software Interface Standard. Der Adapter bildet diese Schnittstelle auf native FreeRTOS-Funktionen ab. ISR heißt Interrupt Service Routine, Interruptbehandler. Im eingesehenen Cortex-M-Pfad wird der Handlerkontext über das IPSR, Interrupt Program Status Register oder Interruptprogrammstatusregister, und gegebenenfalls weitere Portprüfungen erkannt. Thread Mode hat keine aktive Exceptionnummer; Handler Mode bezeichnet den Ausnahmebehandlungskontext.
Bei osMessageQueuePut aus einer erlaubten ISR verlangt der Adapter Timeout null und ruft die native Queue-FromISR-Funktion auf. FromISR kennzeichnet die für diesen Kontext vorgesehene Variante. Ein erfolgreiches Einreihen kann einen höher priorisierten wartenden Thread bereit machen; der Adapter fordert dann den nötigen Wechsel an. Diese interne Kontextentscheidung ersetzt weder die Prioritätsprüfung noch das Funktionsverbot für Mutex-Aufrufe. Im Labor bleiben alle Objekte und Aufrufe bei CMSIS-RTOS2; ein zusätzliches manuelles Yield, also eine Wechselanforderung, wird nicht ergänzt. Das Verständnisziel ist die Verantwortung des Adapters. Die lokale cmsis_os2.c muss den beschriebenen Vertrag tatsächlich implementieren.

Folie 38 / 50

Vertiefung: die native ISR-Schnittstelle

```
/* Vergleich für ein eigenes natives FreeRTOS-Objekt. */
BaseType_t higher_woken = pdFALSE;
BaseType_t ok = xQueueSendFromISR(native_q, &msg, &higher_woken)
/* ok prüfen: erfolgreich oder voll. */
portYIELD_FROM_ISR(higher_woken);
```

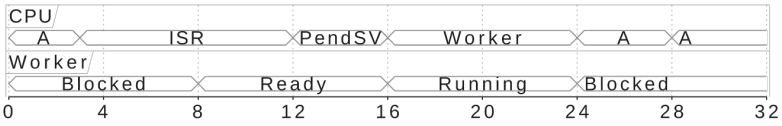
Vergleichscode; das Labor verwendet CMSIS-RTOS2.

Notizen 38 / 50

Der C-Ausschnitt vertieft die native FreeRTOS-Schnittstelle. Er ist Vergleichscode und wird nicht in die CMSIS-Laboraufgabe übernommen. CMSIS-RTOS2 ist die zweite Betriebssystem-Schnittstelle des Common Microcontroller Software Interface Standard. ISR steht für Interrupt Service Routine, Interruptbehandler. native_q bezeichnet eine mit der nativen FreeRTOS-Schnittstelle angelegte Queue, msg die passende Nachricht. BaseType_t ist der native Porttyp für Status und Flag; pdFALSE bedeutet falsch. xQueueSendFromISR reiht ohne blockierendes Warten ein und kann higher_woken auf wahr setzen, wenn eine höher priorisierte Aufgabe geweckt wird.
ok meldet den Übergabeerfolg und ist vom Wechselbedarf in higher_woken zu unterscheiden. Eine erfolgreiche Nachricht erfordert nicht immer einen Wechsel, etwa wenn der Consumer nicht wartet. Bei mehreren nativen Übergaben im selben Handler wird das einmal initialisierte Wake-Flag nicht zwischendurch auf falsch zurückgesetzt. portYIELD_FROM_ISR fordert den passenden Kontextwechsel an; es lässt keinen Thread mitten in einer weiter aktiven ISR laufen. Ohne nötige Wechselanforderung kann der Start bis zu einem späteren Schedulinganlass warten. Im verwendeten CMSIS-Pfad übernimmt der Adapter diese Aufgabe. Seine Handles werden nicht in native Handles umgewandelt.

Folie 39 / 50

Ready wird erst nach der ISR zu Running



Synthetische Zeitachse in µs; Ereignis bei 0, Workerstart bei 16.

Notizen 39 / 50

CPU steht für Central Processing Unit, Prozessor. ISR bedeutet Interrupt Service Routine, Interruptbehandler. Der Worker ist der verarbeitende Thread. PendSV bezeichnet die ausstehend schaltbare Systemexception, über die der übliche Cortex-M-FreeRTOS-Port den Kontextwechsel ausführt. Die obere Spur zeigt die Prozessorbelegung, die untere den Workerzustand. A bezeichnet die zuvor laufende

Notizen, Fortsetzung 39 / 50

Aufgabe Thread A. Die waagerechte Achse zählt Mikrosekunden, µs. Alle Zeitpunkte sind didaktische Annahmen.
Ein Ereignis tritt bei null auf. Thread A läuft noch bis zum angenommenen ISR-Eintritt bei drei. Der Worker wartet zunächst Blocked. Der Übergabeaufruf macht ihn bei acht Ready, also ausführungsbereit. Die ISR führt ihren Rest bis zwölf zu Ende. Danach beansprucht der angenommene Übergang mit PendSV vier Mikrosekunden. Bei sechzehn wird der Worker Running und arbeitet bis vierundzwanzig, bevor er wieder wartet. Ein hoher Threadvorrang unterbricht die noch aktive ISR nicht. Der verlängerte rechte Rand nach vierundzwanzig macht den abschließenden Wartezustand lesbar; er fügt keinen weiteren Verarbeitungsauftrag hinzu. Die Spur setzt voraus, dass keine weitere höher dringliche Exception und kein anderer höher priorisierter Thread den dargestellten Wechsel verzögert. Sie ist keine garantierte H755-Latenz.

Folie 40 / 50

Vier Zeitgrößen und eine Abschlussfrist

Für die synthetische Spur; Abschluss bei 24 µs:

Größe	Rechnung
Ereignis bis ISR-Eintritt	$3 - 0 = 3$ µs
ISR-Dauer	$12 - 3 = 9$ µs
Ready bis Workerstart	$16 - 8 = 8$ µs
Ereignis bis Abschluss	$24 - 0 = 24$ µs

Relative Deadline $D = 25$ µs: Modell erfüllt, Restbudget 1 µs.

Notizen 40 / 50

µs bezeichnet Mikrosekunden. Alle Zeitangaben beziehen sich auf dieselbe Ereigniszeit null. ISR bedeutet Interrupt Service Routine, Interruptbehandler. Ready ist ausführungsbereit; Workerstart ist der erste laufende Befehl des verarbeitenden Threads. D ist die relative Abschlussfrist ab dem Ereignis. Die Zahlen stammen aus derselben synthetischen Spur, nicht aus einer Hardwaremessung.
Die Eintrittslatenz ist drei minus null gleich drei Mikrosekunden. Die ISR-Dauer beträgt zwölf minus drei gleich neun. Ready bis Running ist sechzehn minus acht gleich acht. Die gesamte Antwort bis zum geforderten Abschluss beträgt vierundzwanzig minus null gleich vierundzwanzig. Bei einer relativen Deadline von fünfundzwanzig bleibt eine Mikrosekunde Restbudget. Die acht Mikrosekunden Ready-Warten überlappen mit einem Teil der ISR und dem Übergang. Sie werden nicht zusätzlich auf Eintrittslatenz plus ISR-Dauer plus Übergang plus Workerarbeit addiert. Das würde Zeit doppelt zählen. Der Workerstart allein bei sechzehn beweist keinen rechtzeitigen Abschluss. Zusätzliche Interferenz kann das knappe Restbudget überschreiten.

Folie 41 / 50

Flanke, Callback und Betätigung sind verschiedene Zählungen

- Tasterprellen kann mehrere elektrische Flanken erzeugen
- Hardware-Pendingbit kann Ereignisse zusammenfassen
- Volle Queue kann erfasste Callbacks verwerfen
- Ereigniszeitfilter bildet einen eigenen Verarbeitungsvertrag
- Sequenzlücke beweist nicht jede verlorene Flanke

Ein Filter im Worker schafft keine zusätzliche Queuekapazität.

Notizen 41 / 50

Eine Flanke ist ein Wechsel des elektrischen Pegels. Tasterprellen sind schnelle Wechsel beim mechanischen Betätigen oder Loslassen. Ein Callback ist ein vom Treiber aufgerufener Softwarebehandler. Die Anzahl physikalischer Flanken, ausgeführter Callbacks, übernommener Queue-Nachrichten und als Betätigung akzeptierter Datensätze kann unterschiedlich sein. Ein Hardware-Pendingbit hält häufig nur fest, dass eine Anforderung ansteht; es ist kein beliebig tiefer Ereignisspeicher.
Die aktuelle Übung nummeriert die erreichten Buttoncallbacks vor ihrem Queue-Put. Eine volle Queue verwirft die neue Nachricht und wird getrennt gezählt. Der Worker verwendet eine einfache Unterrichtsregel: Das erste empfangene Ereignis wird angenommen; weitere müssen mindestens vierzig Kernel-Ticks Abstand zum letzten angenommenen Ereignis besitzen. Verglichen werden die erfassten Zeitpunkte, nicht die spätere Bearbeitungszeit. In der vorbereiteten Basis entsprechen tausend Ticks pro Sekunde nominell einer Millisekunde je Tick.
Diese Regel ist ein begrenzter Ereigniszeitfilter. Prellflanken können die Queue füllen, bevor der Worker sie aussortiert; länger auseinanderliegende Prellfolgen können trotzdem mehrfach angenommen werden. Die anschließenden LED-Wartezeiten von sechzig und hundert Ticks stoppen die Hardwarequelle ebenfalls nicht. LED bedeutet Light Emitting Diode, Leuchtdiode. Der Versuch zeigt die verschiedenen Zählgrenzen und beansprucht keine vollständige physikalische Entprellung.

Folie 42 / 50

Laborbasis: eigene Funktionen im vorbereiteten Projekt

- NUCLEO-H755ZI-Q: B1/PC13 und LD1/PB0
- CubeMX und VS Code mit STM32 Extension
- CM4: FreeRTOS mit CMSIS_V2
- Basis: `lab05_button.c` und `.h`
- Eigene Funktionen: `button_student.c`

CM7-Grundstart bleibt Teil der vorbereiteten Basis.

Notizen 42 / 50

NUCLEO-H755ZI-Q ist das vorgesehene Board mit dem Mikrocontroller STM32H755ZIT6. B1 ist der

Notizen, Fortsetzung 42 / 50

eingebaute Benutzertaster auf PC13, LD1 die eingebaute Leuchtdiode auf PB0. CM4 und CM7 bezeichnen Cortex-M4 und Cortex-M7. STM32CubeMX erzeugt die Projektkonfiguration. VS Code bedeutet Visual Studio Code; die STM32 Extension integriert die STM32-Werkzeuge und die CMake-Ziele. CMake beschreibt Übersetzung und Verknüpfung des Projekts. CMSIS bedeutet Common Microcontroller Software Interface Standard; CMSIS_V2 ist die Cube-Auswahl seiner zweiten Betriebssystem-Schnittstelle für den FreeRTOS-Kernel.
Die Lehrperson stellt eine geprüfte Basis mit CM7-Grundstart und CM4-Kernel bereit. Die eigenständige M4-Konfiguration aus Übung 3 wird dafür nicht unverändert übernommen. Das Modul lab05_button.c mit seinem Header stellt Queue, Starthelfer und Einbindung bereit. In button_student.c schreiben Sie selbst Lab05_QueueButton für die Interruptübergabe und Lab05_PulseWorker für die Verarbeitung. Die Startdatei enthält noch keine fertige Lösung; ihr Worker wartet zunächst nur. Der frühere Demonstrator lab05_messages.c gehört zu einer getrennten Zusatzvariante und wird nicht gleichzeitig eingebunden. Die eigene Anwendung benötigt ausschließlich Board und USB-Verbindung zum vorhandenen ST-LINK, dem integrierten Programmier- und Debugadapter. Ein erfolgreiches Übersetzen ersetzt keinen tatsächlichen Boardtest.

Folie 43 / 50

CubeMX: Basis für genau einen Worker

- B1/PC13: `LAB05_BUTTON`, bestätigte steigende Flanke
- LD1/PB0: `LAB05_LED`, Ausgang LOW
- CM4: 1000 Hz Tickfrequenz, Präemption
- Helper erzeugt einen Worker mit `osPriorityAboveNormal`
- Keine zusätzliche Workeranlage in CubeMX

Startstack: 1024 Byte; tatsächlichen Speicherbedarf prüfen.

Notizen 43 / 50

STM32CubeMX erzeugt die Hardware- und Projektkonfiguration. B1 ist der eingebaute Taster auf PC13; LD1 die Leuchtdiode auf PB0. Die User Labels LAB05_BUTTON und LAB05_LED erzeugen die vom Modul benötigten Pin- und Portnamen. LOW bedeutet anfänglich ausgeschaltet. Die steigende Tasterflanke passt nur zum bestätigten aktiven-HIGH-Aufbau, bei dem eine Betätigung einen hohen Pegel erzeugt. Pinbelegung, Polarität und Boardrevision werden anhand des tatsächlichen Boards geprüft.
CM4 bezeichnet den Cortex-M4. Sein FreeRTOS-Kernel verwendet Präemption und tausend Ticks je Sekunde, also 1000 Hz. Der bereitgestellte Starthelfer erzeugt dynamisch eine Dreier-Wertqueue und genau einen Worker mit der CMSIS-Priorität osPriorityAboveNormal. In CubeMX wird dieser Thread nicht nochmals angelegt. CMSIS ist der Common Microcontroller Software Interface Standard. Der Startstack von 1024 Byte ist eine Ausgangsgröße, kein nachgewiesenes Maximum; auch Heap, also dynamischer Speicher, Kernelobjekte und weitere Systemtasks benötigen Platz.
Die EXTI-Anbindung, externe Interruptanbindung, gehört ausschließlich dem CM4. Der IRQ, Interrupt Request oder Interruptanforderung, bleibt bis zum kontrollierten Betriebsbeginn deaktiviert. Seine Gruppierung und logische Priorität müssen die zuvor erklärten Kernelaufrufbedingungen erfüllen. Die genaue Integration steht in der Projektanleitung; der folgende Ablauf beschreibt die Startreihenfolge.

Folie 44 / 50

Starthelfer erzeugt Queue und Worker

1. GPIO und Bootbasis fertig; Button-IRQ gesperrt
2. Vorhandenes `osKernelInitialize()` ausführen
3. `Lab05_ButtonStart()` genau einmal aufrufen
4. Vorhandenes `osKernelStart()` ausführen
5. Vorbereiteter Worker aktiviert den IRQ nach Kernelstart

Helper: Dreierqueue und genau ein Thread.

Notizen 44 / 50

GPIO steht für General Purpose Input/Output, allgemeiner Ein- und Ausgang. IRQ heißt Interrupt Request, Interruptanforderung. Der Buttoninterrupt bleibt während Boot und Objektanlage gesperrt, damit kein Callback einen ungültigen Queue-Handle benutzt. Ein Handle ist der Bezeichner eines Kernelobjekts. Das bereits vorhandene osKernelInitialize initialisiert den Kernel. Anschließend wird Lab05_ButtonStart genau einmal im Zustand osKernelReady aufgerufen, also nach Kernelinitialisierung und vor Kernelstart. Der Starthelfer prüft den Zustand sowie die Tickfrequenz und legt eine Queue mit drei Elementen vom Typ Lab05_Press an. Dieser Typ enthält Sequenznummer und Ereignistick. Der Helper erzeugt auch den einzigen Worker; zusätzliche CubeMX-Threads für diese Aufgabe sind nicht vorgesehen. NULL bezeichnet hier ein nicht angelegtes Kernelobjekt. Ein solches Ergebnis oder ein zweiter Startaufruf führt in den vorgesehenen Fehlerpfad. osKernelStart startet den Scheduler und kehrt bei Erfolg nicht zum Bootcode zurück.
Erst der vorbereitete Workerrahmen behandelt die alten PC13- und NVIC-Pendingzustände und aktiviert EXTI15_10. NVIC ist der Nested Vectored Interrupt Controller, EXTI die externe Interruptanbindung. Ereignisse vor diesem Betriebsbeginn werden im Übungsvertrag bewusst verworfen. Danach ruft der Rahmen Ihre Funktion Lab05_PulseWorker auf. Diese Funktion bleibt im Threadbetrieb und darf nicht zurückkehren. Ein gesonderter Park-Semaphor und ein Software-Producer werden für die aktuelle Aufgabe nicht angelegt.

Folie 45 / 50

Eigene ISR-Funktion: erfassen und übergeben

`Lab05_QueueButton()` in `button_student.c`:

- Sequenz vor dem Put fortschreiben
- Ereigniszeit mit `osKernelGetTickCount()` erfassen
- `Lab05_Press` ohne Warten einreihen
- Erfolg, volle Queue und sonstigen Fehler getrennt zählen

Die ISR wartet nicht und bedient keinen LED-Puls.

Notizen 45 / 50

ISR bedeutet Interrupt Service Routine, Interruptbehandler. Lab05_QueueButton ist die von Ihnen zu implementierende Funktion im erlaubten Callbackpfad. Put bedeutet Einreihen. Der gemeinsame Typ Lab05_Press enthält seq, die fortlaufende Nummer des erfassten Angebots, und tick, den Kernel-Tick beim Erfassen. osKernelGetTickCount liefert diesen Zählerstand. Die geprüfte Basis verwendet 1000 Ticks je Sekunde. Die gespeicherte Zeit kennzeichnet den Callback und nicht jede elektrische Tasterflanke.
Die Sequenz wird vor dem Übergabeversuch fortgeschrieben. Dadurch können verworfene Nachrichten später als Lücke erkennbar bleiben. Für die kleine Wertnachricht genügt ein lokales Datensatzobjekt während des Put-Aufrufs: Die Queue übernimmt bei Erfolg eine Kopie. osMessageQueuePut erhält die vorbereitete Queue, die Datensatzadresse sowie Nachrichtenvorrang null und Timeout null. Timeout null erlaubt kein Warten auf Platz. Die erlaubte Funktion und die gültige IRQ-Priorität sind trotzdem gesonderte Voraussetzungen; IRQ heißt Interrupt Request.
osOK zählt als Annahme, osErrorResource als volle-Queue-Drop, also verworfene neue Nachricht. Andere Statuswerte werden als technische Fehler getrennt gezählt. Die vier IRQ-Diagnosezähler haben nur diesen einen Interruptpfad als Schreiber. In dieser Funktion werden keine Warteaufrufe, lange Berechnungen oder LED-Pulse ausgeführt. LED bedeutet Light Emitting Diode, Leuchtdiode. Die tatsächlich auszuführenden C-Anweisungen schreiben und erklären Sie selbst.

Folie 46 / 50

Eigener Worker: Ereigniszeit und LED-Puls

`Lab05_PulseWorker(argument)`:

- Blockierender Get; Rückgabestatus prüfen
- Erstes empfangenes Ereignis annehmen
- Abstand zum letzten angenommenen Ereignis unter 40 Ticks: ignorieren
- Sonst LD1 an, `osDelay(60U)`, aus, `osDelay(100U)`

API-Fehler im Thread über `Lab05_ButtonCheck(status)` prüfen.

Notizen 46 / 50

Der Worker ist der verarbeitende Thread. Lab05_PulseWorker wird vom vorbereiteten Startmodul aufgerufen. Get entnimmt einen kopierten Datensatz; osWaitForever bedeutet Warten ohne eigene Frist, bis eine Nachricht verfügbar wird. Erst ein erfolgreicher Rückgabestatus erlaubt die Verarbeitung. API bedeutet Application Programming Interface, Programmierschnittstelle. Lab05_ButtonCheck ist der bereitgestellte Fehlerprüfer für Threadaufrufe; er bricht bei einem Status ungleich osOK ab.
Die Unterrichtsregel verwendet die gespeicherten Ereignisticks. Das erste empfangene Ereignis wird angenommen. Danach zählt der Abstand zum letzten angenommenen Ereignis. Beispiel: Bei Zeitmarken 100, 125 und 145 wird 100 angenommen, 125 wegen Abstand 25 ignoriert und 145 wegen Abstand 45 wieder angenommen. Ein ignoriertes Ereignis verschiebt den Vergleichsanker nicht. Die spätere Bearbeitungszeit ist dafür ungeeignet, weil Queuewarten die Abstände verändern kann. Der Vergleich erfolgt mit vorzeichenloser 32-Bit-Differenz für kurze Versuche ohne unerkannte vollständige Tickumläufe. LED bedeutet Light Emitting Diode, Leuchtdiode. Für jede Annahme wird die eingebaute LD1 eingeschaltet. Nach osDelay mit sechzig Ticks wird sie ausgeschaltet; weitere hundert Ticks begrenzen die Bedienrate. osDelay blockiert den Worker und reserviert keine exklusive Prozessorzeit. Phasenlage und Scheduling

Notizen, Fortsetzung 46 / 50

erlauben daraus keine exakte physikalische Pulsdauer. Auch diese Warteaufrufe werden über Lab05_ButtonCheck geprüft. Der einfache Filter garantiert keine vollständige Tasterentprellung.

Folie 47 / 50

HAL-Callback: Pinmaske weiterreichen

Den einen bestehenden Callback ergänzen:

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
    Lab05_ButtonIRQ(pin);
}
```

`Lab05_ButtonIRQ(pin)` prüft Pin und Kernelzustand.

Notizen 47 / 50

HAL bedeutet Hardware Abstraction Layer, Hardwareabstraktion. GPIO steht für General Purpose Input/Output, allgemeiner Ein- und Ausgang, EXTI für die externe Interruptanbindung. Ein Callback ist eine vom Treiber aufgerufene Funktion. Der C-Ausschnitt zeigt die Einbindung in den einen bereits bestehenden HAL_GPIO_EXTI_Callback; es wird keine zweite gleichnamige Funktion angelegt. uint16_t ist ein vorzeichenloser 16-Bit-Typ. pin enthält die Pinmaske der gemeldeten Leitung und wird unverändert weitergereicht.
Die vorbereitete Kette lautet EXTI15_10_IRQHandler, HAL_GPIO_EXTI_IRQHandler, Callback und Lab05_ButtonIRQ mit Pinargument. IRQ bedeutet Interrupt Request, Interruptanforderung. Der Helper reagiert nur auf den konfigurierten Buttonpin und verlangt einen gestarteten Kernel. Danach ruft er Ihre Funktion Lab05_QueueButton auf. Das studentische Modul bildet dort Datensatz, Try-Put und Diagnose; Try-Put bedeutet Einreihen ohne blockierendes Warten.
Im Standardpfad behandelt der HAL-Treiber das EXTI-Pendingflag bereits vor dem Callback. Der Callback quittiert es daher nicht ein zweites Mal. Die Integration wird einmalig in den bestätigten erlaubten Interruptpfad aufgenommen. Der Cortex-M7 darf dieselbe Quelle nicht zusätzlich bedienen. Ein Direktaufruf aus einem Testthread würde den vorgesehenen Kontext verändern. Die Funktion für den LED-Puls bleibt ausschließlich im Worker. LED bedeutet Light Emitting Diode, Leuchtdiode. Die aktuelle Headerdeklaration Lab05_ButtonIRQ(uint16_t pin) und der hier gezeigte Aufruf stimmen überein.

Folie 48 / 50

Diagnose: Nachrichten und Pulse trennen

- Drei Queueplätze; Prellen kann sie vor dem Filter füllen
- IRQ zählt Angebote, Annahmen, Drops und Fehler
- Worker zählt Entnahmen, ignorierte Ereignisse und Pulsbeginne
- Quelle sperren, vollständig abarbeiten, dann CM4 anhalten
- Eine LED-Anzeige ist keine Mikrosekundenmessung

Jeder Diagnosezähler hat genau einen Schreiber.

Notizen 48 / 50

IRQ heißt Interrupt Request, Interruptanforderung. Der Interruptpfad schreibt die Zähler lab05_irq_offered, lab05_irq_accepted, lab05_irq_dropped und lab05_irq_errors: angeboten, in die Queue übernommen, wegen Vollbelegung verworfen oder technisch fehlgeschlagen. Diese Zähler erfassen erreichte Softwareübergaben, keine vollständige Historie elektrischer Flanken. Der Worker schreibt seine eigenen Zähler lab05_worker_handled, lab05_worker_ignored und lab05_worker_pulses: entnommen, nach Zeitregel ignoriert oder Puls begonnen. Das frühere Einreihen und die spätere Verarbeitung sind unterschiedliche Zählgrenzen.
Die Queue besitzt drei Plätze. Prellflanken können diese füllen, bevor der Worker den Vierzig-Tick-Filter ausführt. Die Zahl der Nachrichten muss daher weder der Zahl akzeptierter Betätigungen noch der Zahl physikalischer Tastendrücke entsprechen. Der Pulszähler wird beim Beginn eines Pulses erhöht; er beweist nicht, dass dessen Wartephasen schon abgeschlossen sind. Für einen konsistenten Prüfpunkt wird die Quelle gesperrt und der Worker darf alle angenommenen Nachrichten samt LED-Wartephasen abarbeiten. Wenn er wieder im Get wartet, kann der zuständige Cortex-M4 angehalten werden.
volatile macht die Diagnosefelder für den Debugger zugänglich, ersetzt aber keine allgemeine Synchronisation eines laufenden Mehrfeld-Snapshots. LED bedeutet Light Emitting Diode, Leuchtdiode. Eine leuchtende LED bestätigt einen beobachteten Funktionszustand. Für tatsächliche Interruptdauer oder Startlatenz wären geeignete Zeitmarken und eine bestätigte Zeitbasis nötig. Der kurze Boardversuch liefert ohne solche Aufnahme keine Mikrosekundenwerte und keinen allgemeinen Fristnachweis.

Folie 49 / 50

Quiz: Aussagen mit ihrem Gültigkeitsbereich prüfen

- Kopiert heißt nicht verarbeitet
- Ready heißt nicht Running
- Queueplatz und Payloadbesitz getrennt prüfen
- Nicht wartend heißt nicht automatisch ISR-zulässig
- Größerer Puffer löst keine Dauerüberlast
- ISR sichert Rohdaten, Worker verarbeitet

Notizen 49 / 50

Eine Queue ist eine begrenzte Nachrichtenwarteschlange. Ein erfolgreicher Put bestätigt die Kopie, nicht den Abschluss beim Consumer, dem Empfänger. Ready bedeutet ausführungsbereit und Running laufend. Ein externer Payload, also die eigentlichen Nutzdaten hinter einer Adresse, kann noch vom Consumer benutzt werden, obwohl der Queue-Platz durch Get bereits frei ist. ISR heißt Interrupt Service Routine, Interruptbehandler. Ein nicht wartender Aufruf braucht weiterhin eine erlaubte Funktion und zulässige Interruptpriorität.
Der Interruptpfad sichert bedrohte Rohdaten und behandelt das Hardwareereignis rechtzeitig. Die aufschiebbare Berechnung und erlaubtes Warten gehören in den Worker, den verarbeitenden Thread. Diese Aufteilung verkürzt den sofort belegten Behandlerpfad, beseitigt aber weder Queueüberlauf noch die Gesamtantwortzeit. Das Quiz enthält zehn eigenständige Verständnisfragen; begründen Sie anschließend einen Fehlerpfad oder eine Aufgabenaufteilung.
Durchsatz ist die Anzahl dauerhaft verarbeitbarer Nachrichten pro Zeit. Ein größerer Puffer gleicht eine begrenzte Lastspitze aus, macht aber einen dauerhaft zu langsamen Consumer nicht schneller. Die offene Frage lautet nun, wann der Consumer Prozessorzeit erhalten darf und wie stark seine Ereignisarbeit andere Aufgaben verdrängen kann. Vorlesung 6 führt dafür Server-Verfahren ein: Sie bündeln wartende Aufträge und regeln deren zugelassene Prozessorarbeit. Ein Queueplatz und ein Rechenzeitbudget begrenzen unterschiedliche Ressourcen.

Folie 50 / 50

Quellen und Grenzen der Nachweise

- Buttazzo: *Hard Real-Time Computing Systems*, 3. Auflage
- Arm CMSIS-RTOS2: Message Queue
- CMSIS-FreeRTOS-Adapter
- FreeRTOS: Cortex-M-Interruptprioritäten
- ST: UM2408, RM0399 und lokales STM32Cube-Projekt

Notizen 50 / 50

Giorgio C. Buttazzos Hard Real-Time Computing Systems ist die zentrale Literatur für die Echtzeitmodelle und Ressourcenabhängigkeiten. CMSIS bedeutet Common Microcontroller Software Interface Standard; RTOS2 bezeichnet die zweite Betriebssystem-Schnittstelle. Die Message-Queue-Dokumentation beschreibt Objektanlage, Kopieren, Status und zulässige Interruptkontexte. Der öffentlich eingesehene CMSIS-FreeRTOS-Adapter belegt die dort implementierte Abbildung auf native Funktionen, insbesondere den ignorierten Nachrichtenvorrang und den ISR-Pfad. ISR bedeutet Interrupt Service Routine, Interruptbehandler. Ein veränderlicher main-Branch ist keine fest zugesicherte Version des Laborpakets; die lokale Datei wird zusätzlich geprüft.
UM2408 ist das ST-Benutzerhandbuch der entsprechenden Nucleo-144-Boardfamilie, RM0399 das Referenzhandbuch der betreffenden Dual-Core-Mikrocontroller. Sie sind zusammen mit Boardrevision, Brücken und Projekt die Grundlage für Pins und Interruptzuordnung. Eigene Zahlen in der Vorlesung sind gekennzeichnete Modellannahmen. Renderprüfung, C-Syntaxprüfung, simulierte Protokollprüfung und tatsächlicher STM32-Build beziehungsweise Hardwaremessung sind verschiedene Nachweise. Ohne bereitgestelltes Zielprojekt und Board wird kein erfolgreicher Hardwareversuch behauptet.